Next-Generation AI Compiler Survey and Design Guide

Evidence, architectural choices, and experiments for a performance-first compiler. Forecasts for 2027, 2029, 2031, 2036, and beyond.

Living survey export
Generated: 2026-10-15
A single reading path from technical trends to design decisions, with source appendices.

Design hypothesis. The compiler increasingly becomes an agentic optimization system. Test that hypothesis against strong alternatives. Prioritize runtime performance, then developer productivity and portability. Condition search economics on expected reuse. Treat passes, lowering, representations, and the division between agents and compiler components as choices to evaluate.

Survey narrative

Evidence reviewed through **15 October 2026**. Forecasts cover **2027, 2029, 2031, 2036, and the longer-term future**. This is a living research and design document; forecasts are conditional judgments, not established results.

Central hypothesis: the compiler increasingly becomes an agentic optimization system that chooses strategies, synthesizes implementations and compiler components, evaluates results, and adapts using feedback. Current systems support parts of this direction. They do not establish one required architecture or a permanent boundary between an agent and a conventional compiler.

Design priority: runtime performance first; developer productivity and portability next, with similar importance. Measure search cost from the start. When an improvement will be reused enough to repay its cost, establish performance value before optimizing search expense; for short-lived or latency-sensitive deployment, search cost and payback constrain the design immediately. Cheaper search at comparable runtime is a distinct useful outcome. See the reuse and deployment tradeoff. Correctness, numerical requirements, and deployment constraints define which implementations are acceptable.

Read the purpose and evidence rules first, then the trends and design guidance. The unresolved questions and evidence register explain where recommendations remain uncertain. Detailed source summaries stay in the publication index; this document is the single narrative.

0. Purpose and vocabulary

0.1 What this survey is trying to decide

The survey has two connected purposes: explain the important directions in AI compilation and test a design thesis for the next compiler. The field review must include strong alternatives to agentic methods, otherwise it cannot test that thesis fairly.

An agentic optimizer here does more than enumerate a fixed set of configurations. It uses a learned model to choose or revise optimization actions, synthesize code or transformations, and respond to analysis or execution feedback. It can call conventional search, a solver, a compiler, or an expert library. The architecture should earn its complexity through measured results.

The thesis has three different strengths:

Statement	Assessment	Design implication
Agents can improve selected optimization and compiler-engineering tasks.	Demonstrated in several author evaluations and industry reports.	Treat agents as credible candidates for useful work.
Agentic optimization will become a major organizing principle for some future compilers.	Plausible forecast; adoption and scope remain uncertain.	Build interfaces that allow broader autonomy and replacement of components.
Every future compiler must use an agent, a particular intermediate representation, or a fixed layer structure.	Not established by the reviewed evidence.	Keep competing architectures in the evaluation.

Today’s failures identify challenges and possible research directions. They do not establish permanent limits. Equally, the possibility of a breakthrough does not establish when it will arrive. Forecast direction, timing, and software architecture separately.

The scope includes model graphs, kernels, memory, communication, runtime specialization, compiler construction, controller improvement, deployment, and hardware/software co-design when linked to executable workloads. Coverage is strongest for accelerator tensor workloads. Mobile, embedded, central-processor-only inference, and non-neural machine learning need more coverage before making claims about the entire field. General chip-design automation and generic coding agents are adjacent topics unless they inform a compiler decision.

0.2 Terms used in this guide

Term	Meaning here
AI compiler	A system that translates and optimizes machine-learning programs for execution.
AI-assisted compilation	Using learned models or agents to help optimize programs or build compilers. This overlaps with AI compilation but is a different activity.
Intermediate representation, or IR	A representation of a program used for analysis, transformation, or code generation.
Domain-specific language, or DSL	A language specialized for tasks such as writing tensor kernels.
Pass	A unit of compiler analysis or transformation. A pass need not be handwritten or run in a fixed order.
Lowering	Turning higher-level computation into a representation or executable closer to the target machine. The function does not require today’s sequence of stages.

Control plane	The logic that chooses optimization actions and coordinates experiments.
Controller improvement	Automatically changing reusable optimizer instructions, workflow, or code and evaluating whether future searches improve. It is distinct from improving the current application or the compiler tools.
Compiler substrate	Representations, transformations, analyses, code generation, and tools used by the optimizer. Earlier versions called this the data plane.
Validation contract	The inputs, semantics, numerical or statistical acceptance criteria, quality requirements, and operating conditions an implementation must satisfy.
Oracle	A source of evaluation feedback, such as a reference implementation, a proof checker, or a benchmark. Each has a defined scope and can be incomplete.
Performance portability	Obtaining competitive performance on multiple targets. This can use different target-specific implementations.
Hardware bring-up	Making a new accelerator run the required workloads correctly and then efficiently.

Three uses of **skill** must stay distinct: a vendor's optimization instructions; the Agent Skills packaging specification using `SKILL.md`; and systems that compile skill instructions or manage agent execution state. None of these automatically defines a kernel representation or provides a correctness guarantee. The Agent Skills specification, SIGIL, and SKILL.state cover different parts of that space.

0.3 How to read evidence and forecasts

Evidence label	What it establishes	What it does not establish
Documented capability	A public interface or implementation supports a described mechanism.	A performance advantage or broad deployment.
Author evaluation	The authors report results under a particular protocol.	Independent reproduction or general superiority.
Industry report	An organization reports operational or production experience.	A controlled comparison against every alternative.
Corroborated direction	Different systems show a related capability.	Replication of one numerical result or one architecture.
Design recommendation	A proposed choice for the stated objectives.	A universal requirement.
Forecast	A conditional prediction about future capability or adoption.	An observed result.

The source catalog's relevance tiers answer a different question. **Tier A** directly informs or challenges a design decision; **Tier B** provides infrastructure or adjacent mechanisms; **Tier C** provides context. A non-agent optimizer can be Tier A. A company blog can be highly relevant while still offering limited evidence for generalization. A star in the index marks a source worth prioritizing, not a quality certification.

Count evidence families, not documents. A paper, blog, talk, and repository for one system do not constitute four independent confirmations. Magellan and AlphaEvolve share a lineage. TritorX and KernelEvolve share a Meta setting. Mirage and Prism share researchers. TIRx, the TIRx Harness, KCoral, KDA[2], and CAKE come from one Carnegie Mellon / MLC / NVIDIA research community and count as one family; MetaKernelBench is independently authored but built on TIRx, so treat it as correlated through the toolchain rather than as a member of that family. Record these relationships when assessing confidence.

1. What is changing in AI compilation?

1.1 Two developments are converging

Compilers for AI are gaining richer representations and more control over hardware execution. AI-assisted compilation is gaining better ways to propose, evaluate, and reuse optimizations. Their convergence is credible, but progress in one does not prove that the other must control the whole system.

A useful survey therefore covers improvements to conventional compilation, structured search, learned policies, agentic synthesis, and combinations of these methods. The question for a designer is which combination produces the best acceptable implementation.

1.2 Historical direction

Period	Development	What carries forward
2018-2022	Tensor compilers, schedule search, and reusable compiler infrastructure expand.	Algorithm/schedule separation, cost models, and retargetable representations.

2020-2023	Learned compiler policies and reinforcement-learning environments become practical research tools.	Training and evaluation interfaces for optimization decisions.
2023-2024	Language models begin proposing pass sequences and operating on compiler representations.	Code and compiler knowledge can supply useful search priors.
2025-2026	Kernel agents, synthesized heuristics, structured feedback, and compiler/agent co-design develop together.	Evaluate the optimizer and the environment it can change.

These periods overlap. New methods do not make earlier methods obsolete by definition.

1.3 Six trends that matter for design

Two changes organize these trends: **what the compiler can optimize** and **how it chooses changes**. Expanding scope across kernels, communication, memory, precision, and runtime creates pressure to coordinate decisions; agentic control is one candidate response. Compiler components can also become optimization targets. This is a causal hypothesis to test, not proof that a large search space requires agents: structured search, decomposition, and conventional compiler methods remain alternatives.

Hardware decisions are becoming more explicit

TileLang, Triton Gluon, Triton Low-level Language Extensions, CuTe DSL, FlyDSL, Pallas, CUDA Tile, and TVM TIRx expose different combinations of tiles, layouts, memory spaces, synchronization, and scheduling. Helion provides a higher-level interface with substantial automatic tuning. Their common lesson is to preserve access to decisions that affect performance, while allowing automation to handle details when it performs well.

The design choice is the balance between automation and explicit control. A high-level interface can reduce effort and enable retargeting; a lower-level interface can expose a new hardware capability before an automatic optimizer supports it. Neither choice establishes one universal representation. An independent CUDA Tile evaluation shows the cost of an immature target: the same short attention kernel was the fastest tested implementation on one Blackwell GPU and slower than FlashAttention-2 and Triton on another. See the source summaries for TileLang, Gluon, CuTe DSL, Pallas, and TIRx.

Optimization is expanding beyond isolated kernels

Triton-distributed and its related DITRON compiler coordinate computation and communication. DITRON reports application gains without agentic control, including gains from overlap despite slower individual matrix multiplications. Shardy handles tensor sharding, while Event Tensor addresses dynamic dependencies. These are compiler directions in their own right; an agent is not a prerequisite for studying them.

Design consequence: expand the optimization boundary when communication, memory movement, launch overhead, or runtime scheduling dominates. Do not infer that every workload needs one giant kernel or one controller for an entire fleet.

Testable hypothesis: with comparable performance headroom, feedback, and resources, agents may gain more over strong non-agent methods when decisions interact and existing rules are incomplete. Test mature local-kernel tasks against coupled application tasks, while giving both methods the same action space and information. Measure held-out application gain, failures, and cost. Equal or better non-agent results in the coupled tasks would weaken the hypothesis. A larger space can also make agent search worse; advantage need not increase monotonically with size.

Several search methods remain competitive

Mirage and Prism use structured search and equivalence reasoning. MLGO embeds learned decision policies. Helion's language-model-guided tuner combines model proposals with conventional refinement.

Design consequence: compare methods under the same objective and validation contract. An agent can use these methods as tools; describing the surrounding system as agentic does not establish the value of its reasoning component.

Agentic optimization has useful measured results

KernelEvolve, GEAK, Hyperloom, CAKE, and Ave, previously Argus provide evidence across different workloads and organizations. MaxKernel extends the picture to Pallas on tensor processing units.

The reported results mix kernel speed, application throughput, search efficiency, and operator coverage. Keep these outcomes separate. KernelBench-X finds that 46.6% of correct generated kernels are slower than its PyTorch eager baseline; that is a result for the evaluated methods and protocol, not a future ceiling. Benchmark source. KernelBench-Verified shows how much the protocol matters: with TF32 enabled in the FP32 PyTorch baseline and hidden input transformations, the best single-turn model's mean speedup falls from 1.43 to 0.88 times. In BF16, where the baseline already uses Tensor Cores, the same model reaches 1.63 times on fused-operator problems.

KernelOPT demonstrates integration-aware optimization: preserve vendor libraries and optimize the generated Triton portions of compiled models, with fallbacks included in its KernelBench results. KernelZero explores a different path, improving the code-generation model and task curriculum offline. Compare integrated runtime, kernel speed, training/search resources, and operator coverage separately.

Feedback is becoming more informative

Compiler diagnostics, profilers, reference tests, invariant checks, and equivalence tools can explain why a candidate failed. CAKE supplies localized schedule and hardware findings. Ave checks specified data-flow relationships. AsmEvo checks assembly edits against an existing binary. T-LLM Compiler combines compiler feedback with verification tools.

These checks cover different failure modes. Successful compilation, sampled numerical agreement, bounded verification, and application-quality validation are not interchangeable guarantees. There is no single weakest-to-strongest ordering that applies to all of them.

The compiler itself can become an optimization target

Magellan evolves compiler heuristics. CAKE describes changes to its representations and compiler harness. AsmEvo operates after conventional compilation. Compiler 2.0 and MOCHA provide broader research agendas for compiler construction and retargeting.

Compiler implementation is now a stronger experimental choice. AI as a Compiler directly synthesizes specialized Triton-to-PTX kernels and reports 0.83–3.34× autotuned Triton on selected GPU cases. PTXBench provides important contrary evidence: directly generated target-specific instructions do not consistently match expert libraries across a different H100/B200 workload suite. ACT generates accelerator backends from instruction-set descriptions using equality saturation and constraint programming, without an LLM. The AI-lowering result also shows that the deployable gain depends on the acceptance level: when provable equivalence against the Triton reference is required, the same kernels fall to 0.24× (matrix multiplication), 0.78× (FlashAttention), and 0.77× (convolution), because the verifier covers fewer instructions than the model can use. Each kernel costs dollars rather than milliseconds to produce, so the per-workload-versus-offline reuse condition applies in full. These are competing paths, not a reason to assume all future compilers must be agentic.

Design consequence: allow the system to improve its tools as well as its programs. Validate generated compiler components across workloads, because an improvement for one kernel can create regressions elsewhere. A component authored by an agent can still execute deterministically.

The optimization controller can also become a target. Changing its reusable instructions or search workflow is a different experiment from changing a kernel or compiler pass. GEPA supplies limited kernel-specific prompt-evolution evidence; broader controller-code and recursive-improvement studies are mostly adjacent agent research. This guide now treats controller improvement as an explicit research direction, with its own evaluation rather than an assumed property of every agentic compiler.

1.4 Where to look for evidence

Source community	Useful evidence
Compiler and systems conferences	Mechanisms, evaluation protocols, artifacts, and reproduction studies.
Machine-learning conferences	Learned optimization policies, synthesis methods, and benchmark design.
LLVM, OpenXLA, PyTorch, TVM, and vendor documentation	Supported interfaces, deployment constraints, and changes to compiler infrastructure.
First-party engineering reports	Production workloads and practical integration experience, with attribution limits.
Research agendas and public programs	Plausible future directions; distinguish plans from completed evaluations.

1.5 Public visions and their limits

Compiler 2.0 emphasizes better abstractions and machine learning for building and retargeting compilers. MOCHA connects learned rewrite generation with structured search and checking. Compiler.next extends compilation ideas to applications built around models, prompts, and agents. The New Compiler Stack organizes language-model/compiler interactions.

Use these visions to formulate experiments. Funding, an architectural proposal, or a compelling demonstration does not establish that an approach will become the dominant implementation.

1b. Choosing among conventional, learned, and agentic optimization

The following are design alternatives. A product may use more than one.

Approach	Why choose it?	Main limitation	Evaluation question

Fixed passes and expert libraries	Predictable execution, mature coverage, and strong existing implementations.	A fixed search space can miss workload-specific opportunities.	How much application performance remains beyond the best configured baseline?
Structured search or superoptimization	Explore combinations systematically and exploit known semantics.	Representation and pruning choices restrict the searchable space.	Does it find better implementations within a useful budget?
Learned policies inside the compiler	Make repeated decisions cheaply after training.	Generalization and training-data coverage require maintenance.	Does the policy improve held-out workloads and new compiler versions?
Agent-guided search and synthesis	Use contextual reasoning and generate implementations beyond a fixed parameter grid.	Requires effective feedback; results and cost can vary.	What does adaptive reasoning add beyond the same tools with conventional search?
Agent-generated compiler components	Amortize a useful heuristic or transformation across many compilations.	Compiler changes have a broad regression surface.	Do generated components generalize beyond the search workload?

For performance-first design, retain the best validated candidate found by any method. An agent should not win an architecture comparison simply because it produced more candidates, used more hardware, or changed the numerical contract. Conversely, a costly search can be worthwhile when its result is reused extensively.

A useful first experiment compares a strong conventional pipeline, structured search, an agent restricted to the same actions, and an agent allowed to synthesize new implementations. Report performance at matched budgets and the best attained performance under a larger declared budget. This separates search efficiency, search-space expressiveness, and the value of synthesis.

2. How agents can help

2.1 Mechanisms and appropriate uses

Mechanism	Practical use	What to verify
Propose promising configurations	Seed tuning with workload and hardware context.	Whether proposals improve search efficiency or the final implementation.
Synthesize kernels and transformations	Explore fusion, schedules, algorithms, or low-level code.	The declared semantics and numerical requirements.
Diagnose failed or slow candidates	Use compiler findings and profiles to target the next change.	Whether the diagnosis predicts a measurable improvement.
Choose optimization scope	Move between graph, kernel, communication, and runtime decisions.	Application benefit after integrating all changes.
Evolve compiler components	Generate heuristics, analyses, or backend support.	Regression behavior on a separate workload corpus.
Reuse accumulated experience	Retrieve useful implementations and unsuccessful experiments.	Compatibility with the current hardware, software, and input assumptions.
Improve the controller	Propose reusable instruction, workflow, or controller-code changes.	Better future searches on separate tasks after accounting for development and evaluation cost.

Reasoning Compiler combines language-model proposals with tree search; ACCLAIM explores multiple abstraction levels; CompileIQ exposes internal tuning controls; Magellan evolves executable heuristics. These are different mechanisms, not interchangeable implementations of one proven agent architecture.

2.2 A practical optimization loop

1. Define the workload, performance metric, and validation contract.
2. Capture a strong baseline and profile the complete execution.
3. Choose a promising region or interacting set of decisions.
4. Generate candidates using conventional search, learned policies, an agent, or a combination.
5. Reject candidates that violate the applicable checks; compile or materialize the survivors.
6. Measure candidates on the target and integrate promising ones into the application.
7. Keep improvements that satisfy the contract; retain a usable fallback and an experiment record.
8. Use the findings to improve future proposals. Where justified, open a separate experiment to change the reusable controller or compiler tools.

This is a reference workflow. Cheap static checks can happen before compilation; some checks require execution. Candidate generation and validation can run in parallel when resources allow. The workflow does not require one specific agent topology.

2.3 Boundaries that should be explicit

The optimizer can propose changes to programs, transformations, or evaluation tools. It should not silently redefine the acceptance criteria to make its own output pass. Changing numerical tolerances, supported inputs, or model-quality requirements is a separate design decision.

Record whether a candidate was tested, formally checked within a modeled subset, or validated in an application. Keep evaluation inputs separate from search feedback when measuring generalization. This is especially important when the system can modify its own harness.

2.4 Three targets of improvement

An agent in the control plane can optimize applications without changing its own implementation. Distinguish the artifact that persists after an experiment:

Target	What changes	What must improve
Application	Kernels, fusion, layouts, placement, or the runtime plan.	The accepted application's execution under its contract.
Controller	Reusable instructions, search strategy, retrieval policy, orchestration, or agent code.	The quality or cost of future optimization jobs.
Compiler	Analyses, transformations, representations, or backend implementation.	Compilation outcomes across supported workloads and targets.

For example, trying another tile after a slow measurement changes the current search. Retaining an automatically proposed policy for deciding when to stop tile search and explore fusion changes the controller. Generating a reusable lowering transformation changes the compiler. One agent can perform all three jobs; these are evaluation boundaries, not a requirement for three agents or permanent software layers.

Here, **control-plane self-improvement** means an automatic process proposes, evaluates, and retains a reusable controller change. A separate optimizer may perform that process. Reserve **recursive self-improvement** for the stronger case where the mechanism that proposes improvements also changes. Persistent experience can adapt later behavior without either kind of code rewrite; merely storing logs establishes neither improved behavior nor an improved controller.

Use one rule to classify an edit: a controller change alters the decision procedure — instructions, search strategy, retrieval logic, orchestration, or agent code — not merely the data the procedure has stored. Adjusting stored experience alone is memory adaptation, not controller improvement. A retained decision procedure such as a learned stopping rule can also count as a learned policy in the alternatives comparison; classify it by what persists and evaluate it here whenever the reusable procedure, rather than its data, is what changes.

3. How compilation processes may change

3.1 Keep necessary functions; reconsider their implementation

Function	What must be achieved	What remains optional
Represent computation	Preserve enough information to implement and evaluate the workload.	One common representation, multiple representations, or task-specific forms.
Transform and optimize	Find an acceptable implementation with better performance.	Handwritten passes, learned selection, generated rewrites, or direct synthesis.
Map to hardware	Produce runnable code with the required execution behavior.	Fixed staged lowering, adaptive paths, generated backends, or direct low-level generation.
Validate	Establish confidence under an explicit contract.	The combination of analyses, proofs, tests, simulation, and deployment checks.
Coordinate execution	Respect dependencies and resource constraints.	A conventional runtime, persistent kernels, static schedules, or adaptive policies.

Passes are useful units of implementation and review. They are not a required user-facing abstraction. Lowering describes a function; it does not establish that today's lowering sequence must survive.

3.2 Current challenges and plausible solution paths

Current challenge	Possible path forward	Evidence that would show progress
Fluent code generation can still misunderstand program semantics.	Better representations, training data, analyses, and equivalence feedback.	Higher validated coverage on unseen programs without relaxed contracts.
Correct kernels can still be slow.	Combine profiling, performance models, structured search, and learned proposals.	Better application performance, including failures and regressions in the denominator.
Fusion, asynchronous execution, and numerical behavior are difficult to validate.	Explicit dependencies, numerical contracts, invariant checks, and scoped proofs.	Fewer escaped errors on broader workload and input distributions.

New hardware has little training data.	Use hardware specifications, simulators, compiler traces, and targeted experiments.	Competitive performance and coverage on genuinely new targets.
Search is expensive.	Reuse results, improve proposal quality, learn small policies, and generate reusable heuristics.	Better performance-versus-budget curves.
Changing the compiler can break unrelated workloads.	Separate evaluation sets, component-level checks, reproducible builds, and staged deployment.	Sustained improvements after compiler and workload updates.

Negative results from mlirAgent, LLM4IR, and KernelBench-X motivate these tests. They do not determine the capabilities of models or compilers five or ten years from now.

3.3 A starting architecture with replaceable components

Start with a workload specification, an experiment coordinator, several candidate-generation methods, compilation and analysis tools, evaluation, and a store of accepted artifacts. Keep the interfaces explicit enough to compare or replace each component.

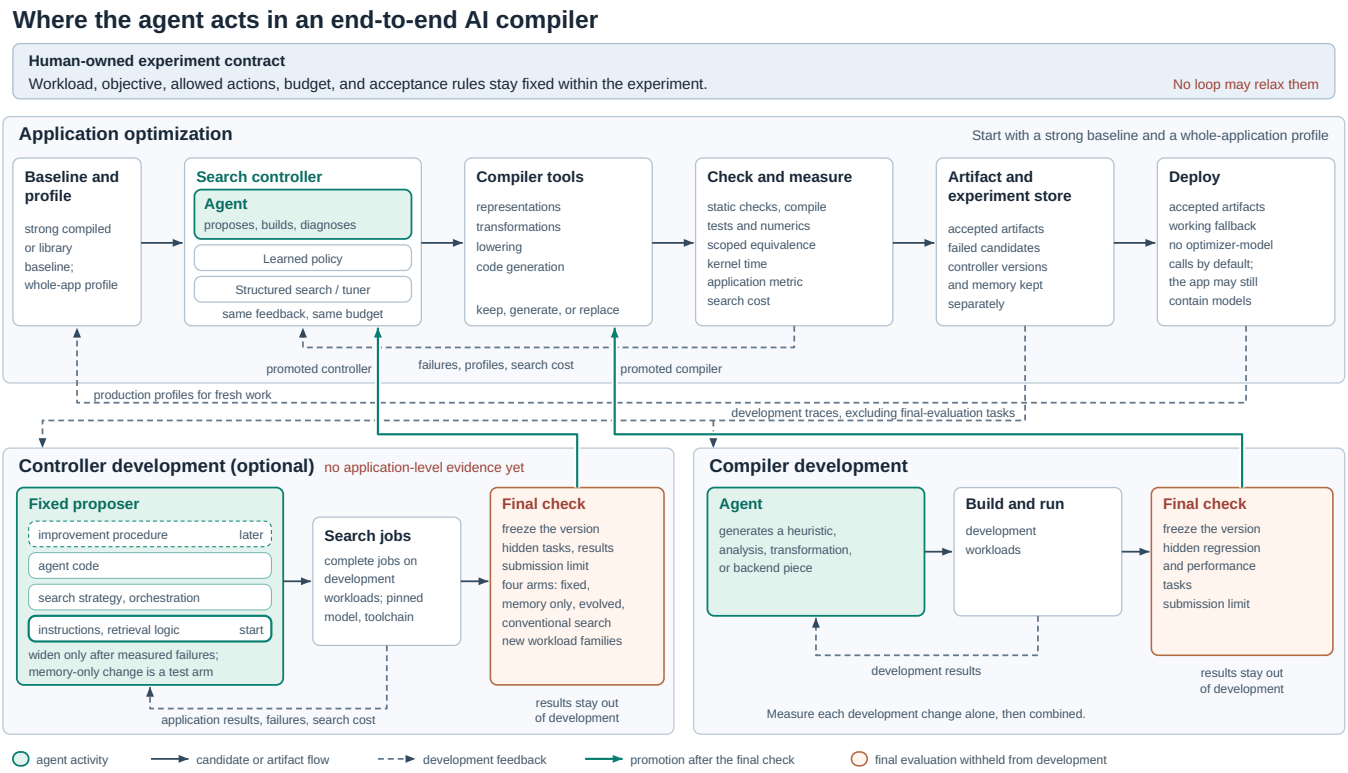


Figure 1. One possible starting architecture. Humans fix the experiment contract. In application optimization, an agent, a learned policy, and structured search propose candidates with the same feedback and budget; checks and measurement decide, and deployment needs no optimizer-model calls by default. Development traces exclude final-evaluation tasks. Compiler development and optional, late-stage controller development each iterate on development feedback, freeze a selected version for a hidden final check, and promote it only if it passes. Controller changes start with instructions or retrieval logic and widen only after measured failures. The three targets do not require three agents or permanent layers. The figure continues with who does what, how the controller improves across optimization jobs, and how far the agent reaches by horizon.

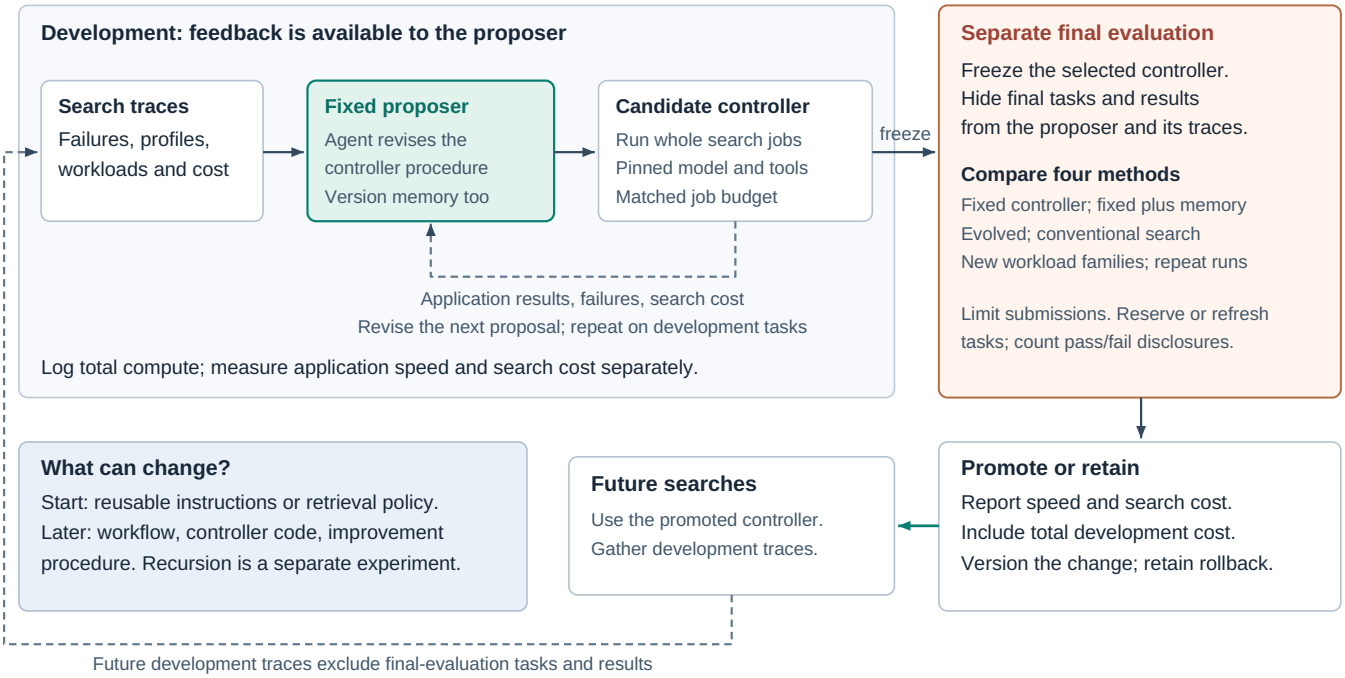
Who does what

Responsibilities, not required agents or layers.

Role	What it may change	What bounds it
Application optimization, per workload	Configurations and scope across graph, kernel, communication, and runtime; synthesized kernels, fusions, and rewrites; diagnosis of rejected or slow candidates; reuse of compatible earlier experiments. Agents, conventional search, and learned policies can all propose.	Validation and target measurement decide. Search cost must pay back within the artifact's useful life. An agent keeps its place only where it beats the alternatives at an equal budget.
Compiler development, offline	Heuristics, analyses, transformations, backend support, and operator coverage for new hardware.	Development workloads supply feedback; a selected version then faces separate regression and performance evaluation, including after compiler updates. Merged components need no optimizer-model calls during normal execution.
Controller development, offline and gated	First a restricted decision procedure, such as instructions or retrieval logic; search strategy, orchestration, or agent code only after restricted changes leave measured failures. Changing the improvement procedure itself is a later experiment.	Late-stage: pursue it after the other two loops deliver repeated value. Compare against a fixed controller, a memory-only variant, and conventional search with pinned model and toolchain versions. No application-level evidence yet.
Fixed for the experiment	Nothing during the experiment: the workload, objective, validation contract, final-evaluation pool, and its submission limit. The deployed runtime runs accepted artifacts and keeps a fallback.	Changing these is a separate design decision, never an optimization action. Final tasks and results stay outside every proposer's access.

How the controller improves across optimization jobs

Optional offline experiment after application search and compiler evolution demonstrate value.



How far the agent reaches, by horizon

Each call has a dated public test. Confidence is qualitative, not a probability.

Horizon	Central forecast	Dated public test	Confidence
2027	Tuning, kernel synthesis, and diagnosis integrated with existing toolchains; conventional backends remain.	23 September 2027: two independent organizations publish integrations with qualifying application gains from agent decisions.	direction high; timing medium
2027 controller checkpoint	Restricted controller improvement becomes testable on compiler workloads.	23 September 2027: a public controller change beats a strong fixed controller and a memory-only variant on two held-out compiler-workload families.	medium for restricted improvement; low for broad recursive improvement
2029	Leading systems coordinate graph, kernel, communication, dispatch, and heuristic decisions.	23 September 2029: a public system jointly searches two decision areas, one involving communication or runtime, and beats strong non-agent joint search.	direction medium-high; timing medium
2031	Some compilers become continuing optimization services; generated components enter default paths.	23 September 2031: a generated analysis, transformation, or lowering component stays in a public toolchain's default path for two releases.	direction medium; timing medium-low
2036	Some platforms repeatedly generate substantial components and explore algorithms, execution, and hardware together.	23 September 2036: two independent toolchains each generate two kinds of component, and one system jointly searches a hardware design parameter and an execution strategy.	direction medium-low; timing low
Beyond	Synthesis from workload intent and deployment requirements.	23 September 2039 precursor: one specification and agent policy applied to two hardware families, one withheld during development.	direction low; timing unassigned

Humans establish the workload, objective, and validation contract for an experiment. Application candidates can come from agents, conventional search, learned policies, or a combination. Compiler development repeatedly builds and evaluates proposed toolchain changes on development workloads; a selected version then faces separate regression and performance evaluation. Controller development repeatedly runs complete application-optimization jobs with proposed controller versions. Neither development loop receives final-evaluation tasks or results as proposal feedback. Acceptance rules stay fixed during the experiment; changing those rules is a separate design decision.

In this starting design, there are **no optimizer-model calls during normal execution**. The deployed application can itself contain models, and its execution need not be deterministic. Retain a working fallback and use production profiles to identify fresh optimization work. Each component can be retained, generated, merged, or replaced as described in the architecture choices.

For example, initially use an existing backend to implement a generated schedule. If backend limitations repeatedly prevent useful schedules, evaluate a new lowering path or a generated backend component against the existing path. If a direct assembly optimizer improves the final artifact, include it as another candidate-producing stage. Decide from measured behavior rather than from a rule that agents must always remain above the backend.

4. Gaps that guide the research agenda

These gaps concern evidence, capability, and deployment. They are challenges to investigate, not permanent restrictions on the future architecture.

4.1 Application-level evidence

Kernel and pass-level gains are abundant relative to controlled application comparisons. KernelEvolve reports production throughput improvements; Hyperloom and GEAK describe application optimization loops. Their settings differ from public benchmark suites.

Design guidance: select representative training or serving workloads before optimizing. Fix the baseline, precision, quality criteria, hardware, and workload mix. Attribute gains from kernels separately from framework, batching, caching, or runtime changes. Long-running production evaluation strengthens deployment confidence, but is not a prerequisite for a useful research result.

Profile the **addressable fraction**: the time that the permitted optimization actions can actually change. The kernel-headroom study is an instructive, narrow example: its table projects 1.32–2.49% transformer time savings on selected A100 profiles. Its 58.2% recommender figure is the total addressable share; one embedding kernel is 37.4%. These are not KernelEvolve’s private workloads, so the study cannot explain that system’s reported gains directly.

Illustrative calculation: if an affected region occupies 10% of serial runtime and becomes twice as fast, total runtime falls by 5%; application speedup is about 1.053 times. This assumes other work is unchanged. An 80–86% library share alone therefore does not imply a 1% ceiling. Library optimality, achievable kernel gains, and the allowed actions are additional assumptions. With overlapping execution, measure effects on the critical path instead of adding overlapping kernel durations. Fusion, communication changes, or algorithm changes can alter the addressable fraction itself.

4.2 Correctness and numerical behavior

Check	What it can establish	Important limit
Compiler checks and known transformations	Structural validity and properties enforced by the implementation.	A compiler can contain bugs; generated transformations need their own justification.
Reference and operator tests	Agreement on tested inputs and argument combinations.	Finite tests do not cover all executions.
Numerical and model-quality checks	Compliance with declared error and application-quality requirements.	A tolerance can be inappropriate for a different model or use.
Invariant and equivalence checking	Properties within the modeled language, assumptions, and checking scope.	Arithmetic models, concurrency, bounds, and invariant completeness matter.
Application replay and staged rollout	Behavior under representative execution and production conditions.	Rare inputs, drift, and interactions can still be missed.
Stochastic or nondeterministic execution	Repeated trials estimate error and quality distributions under specified device states; check confidence bounds and tail requirements.	Finite samples and imperfect noise models can miss rare failures, drift, or untested conditions.
Contract-evasion checks	Fresh seeds and held-out input distributions per scoring run, a written specification of legal inputs, replay with new tensors after timing, stress probes at boundary conditions, real application traces, and an evaluator outside the optimizer’s environment.	Catches candidates that improve the score without improving the workload; cannot prove the absence of untested evasions.

Combine checks according to the allowed action. A configuration change, a new reduction order, and a generated backend require different evidence. Ave's data-flow assertions and CAKE's schedule checks are useful examples of targeted feedback, not complete proofs for arbitrary accelerator programs.

Stochastic hardware can still have a deterministic mathematical reference; what changes is the acceptance rule. Specify repeated-trial sampling, allowed error or quality loss, failure probability, and operating conditions. IBM's analog-hardware evaluation tutorial already treats variation and drift explicitly. Model-quality checks can therefore be extended to statistical contracts; their meaning is not undefined.

Measuring the Checker makes the benchmark oracle itself a measurable component. The official KernelBench checker misses 16.9% of 7,384 independently witnessed synthetic faults across 188 tasks; this is not a production error rate. KDA[2] shows what the contract-evasion row catches in practice: candidates claiming 3.74×, 3.57×, and 5.16× were rejected for a hard-coded normalization constant, overflow to NaN on every real case, and illegal history truncation; the accepted result was 2.96×. Use injected faults, held-out inputs, and false-rejection tests to assess acceptance machinery before trusting an agent's speedup.

4.3 Search cost and reproducibility

Track model calls, tokens, compilation time, device time, failed trials, and elapsed optimization time. Preserve the accepted artifact, its dependencies, and enough records to rerun the measurement. Replaying an accepted artifact is different from reproducing the exact stochastic search trajectory.

For reusable improvements, establish attainable performance before concentrating on cheaper search. For deployment-specific search, check its latency and payback at the outset. Without cost measurement, an architecture comparison can confuse a better method with a larger budget. Cache results only when the input assumptions, target, compiler, and validation contract remain compatible.

A simple decision rule compares search, integration, and maintenance cost with the expected saving over all executions before the artifact becomes invalid. Use one declared accounting unit and separately enforce wall-clock deployment deadlines; tokens, device time, and engineer time are not interchangeable quantities. Count failed attempts as part of the cost of an accepted artifact. One search per deployment can still serve millions of executions, while fleet-wide heuristics still require maintenance. Report campaign totals separately from per-artifact costs.

Helion illustrates a useful alternative outcome: cheaper tuning at roughly comparable final latency. Under an agreed performance tolerance, that is a search-efficiency improvement even without a runtime gain. Keep the two results separate and check per-shape regressions; the tolerance must be chosen before evaluation.

Shared evaluation infrastructure is part of search cost. KCoral reports 2.58× more **GPU evaluations per minute** than a local sequential baseline on one B200, through remote execution and CPU/GPU overlap. This is not a kernel speedup and excludes model generation. Measure GPU occupancy, timing fidelity, queueing, isolated test time, and failure recovery.

4.4 Interoperability across compiler stacks

A portable experiment interface can describe workloads, candidate actions, results, and artifact identity even when each backend uses a different language. This can reduce integration effort without assuming a universal executable representation.

The alternative, a more unified representation, remains worth testing. Compare expressiveness, target performance, validation effort, and maintenance cost. Existing fragmentation is evidence about today's ecosystem, not proof that future consolidation cannot work.

4.5 Interfaces that expose hardware-relevant decisions

Expose layouts, synchronization, memory use, available instructions, and profiling results where they affect performance. Prefer structured information where it prevents ambiguity; retain source-code and diagnostic access where a rigid schema would hide important details.

The open question is how much structure helps the optimizer. An interface that makes experiments reliable is valuable; an interface that excludes the winning implementation limits attainable performance. Compare both restricted and wider action spaces under the same evaluation.

TIRx Harness distinguishes a compiler intermediate representation from the surrounding agent environment: a kernel programming face, synchronization and race checks, numerical simulation, documentation and reference kernels, profiling, and evaluation services. Selected Blackwell attention-family kernel results are promising, but do not isolate the benefit of each component, and the harness, KCoral, KDA[2], and CAKE are one research community rather than independent confirmations. One measured detail is useful on its own: with the same agent and acceptance contract, KDA[2] reached 2.85–2.96× across three kernel languages, so for that operator the language mattered less than the surrounding environment. **Test:** compare a conventional compiler interface, tools alone, tools with references, and the complete harness using fixed models, workloads, and budgets. Report incomplete checker verdicts instead of treating them as success.

4.6 The optimizer's own workflow and memory

The controller is now a research target in its own right. The evidence supports several mechanisms with different scopes; it does not establish a single self-evolving compiler architecture.

Mechanism	Relevant evidence	What remains unproven for this compiler

		design
Evolve reusable instructions	GEPA changes prompts used by kernel-generation agents.	Application benefit and transfer of the learned controller to unseen workload families under comparable total budgets.
Adapt through persistent experience	KOPE describes kernel-search memory with a fixed model.	Full-text protocol verification is pending; memory adaptation does not establish autonomous workflow rewriting.
Search workflow or harness code	Automated Design of Agentic Systems, AFlow, and Meta-Harness generate agent designs or harnesses.	Their broader-task results need compiler-specific evaluation; the outer optimizer need not change.
Let agents edit their own implementation	Self-Improving Coding Agent and Darwin Gödel Machine evaluate agent-code changes.	Coding-task gains do not establish better compiler-search decisions.
Change the improvement procedure itself	Hyperagents makes the agent-modification procedure editable.	Compiler transfer and sustained gains remain open; its main experiments still fix parts of selection and evaluation.

Assessment: there is direct but narrow kernel evidence for instruction improvement, adjacent evidence for broader controller redesign, and an abstract-only kernel-memory lead. Full compiler-controller self-evolution is a hypothesis to test. Automated Design of Agentic Systems, Darwin Gödel Machine, and Hyperagents share a research lineage; Meta-Harness and GEPA also have author overlap. Count these dependencies instead of treating every paper as independent confirmation.

No application-level evidence yet. Note the ceiling on today’ s direct evidence: the strongest controller-improvement result is a hardware-utilization proxy on a single target and model, not a runtime or application measurement. No study yet shows a retained controller change improving end-to-end application performance in a compiler under a fixed contract. Treat control-plane self-improvement as unproven at the application level for this design, not merely as under-replicated.

FlowCompile provides a structured workflow-optimization alternative. AgentFlow supplies analysis, while Auto and skill-compilation research explore reusable execution artifacts. DeepSeek Harness and SKILL.state provide runtime and state-management mechanisms. Such infrastructure can support an evolving controller, but analyzing, compiling, or hosting a workflow does not demonstrate that it improves itself. AFlow, AgentFlow, and FlowCompile are distinct systems.

The main gap is a controlled compiler experiment. A persistent controller change should improve fresh searches, not merely replay a successful kernel or benefit from more accumulated device time. Version the controller and its memory snapshot separately from executable artifacts and compiler components. Keep search traces for diagnosis and evaluation tasks separate from those traces. Test stale or misleading memories after workload and hardware changes, and measure the cost of maintaining the controller as well as running it.

MetaKernelBench transfers distilled optimization advice between CuTe DSL and TIRx on the same 74 paired problems. Across six models, reported matched-target-attempt lift ranges from −19% to +29%, with negative transfer. Source-experience development cost is additional. Regressions occur on 16–45% of problems for every model and direction. D2K-Bench measures the other direction: when expert design guidance is supplied, the three frontier agents correct on all 26 tasks move from 1.69× to 2.49× geometric-mean speedup on B200, and correctness rises from 93.1% to 98.5%. Knowledge supplied to the controller helps far more consistently than knowledge the controller distills for itself, and much of the remaining gap is design discovery rather than implementation. Distinguish retained examples, memory-only adaptation, skill instructions, learned policies, and changes to the controller procedure; none is automatically beneficial on fresh tasks.

4.7 Training and evaluation data

ComPile and Meta LLM Compiler provide compiler-representation data. KernelBook, TritonRL, DRTriton, and AMDKernelVault provide kernel-generation or training resources. Their data sources and validation differ.

Record failed compilations, numerical errors, and correct-but-slow candidates as well as successes. Separate training data, optimization feedback, and final evaluation. Test transfer across models, shapes, hardware, and compiler versions. More samples of one familiar representation do not by themselves demonstrate broad portability.

4.8 Human review and maintenance

Assign ownership to accepted compiler changes and generated kernels. Measure review effort and escaped defects, not only the number of generated patches. Compiler-specific tools such as Archer and llvm-harness provide relevant evidence; generic code-review automation alone is weaker evidence for semantic compiler changes.

Automate acceptance only for action classes with adequate checks. Larger changes can become more autonomous as the evaluation improves. Human review requirements are a deployment choice and current confidence measure, not a permanent limit on synthesis.

4.9 Security and artifact provenance

Run generated candidates with appropriate isolation, preserve where released artifacts came from, and control which tools and files optimization jobs can change. Protect proprietary workloads and hardware documents according to their actual access requirements. These are implementation responsibilities for deployed optimization systems, not evidence for a specific compiler layer structure.

4.10 Comparable benchmarks

Use a progression from compiler transformations to individual kernels, fused regions, complete applications, and multi-device execution. Include new-hardware enablement where relevant. KernelBench, KernelBench-X, KernelBench-Verified, FlashInfer-Bench, JAXBench, KernelGenBench, PTXBench, and llvm-bench each cover part of this space. Two newer benchmarks measure the evaluation itself rather than the kernels: Measuring the Checker quantifies what an oracle misses, and D2K-Bench separates design discovery from implementation by running each task with and without expert guidance. PeepholeBench extends the progression to compiler-engineering tasks and scores profitability, not only validity.

For every result, record the denominator, validation scope, workload coverage, target, software versions, budget, and variability. Configure the baseline as practitioners would deploy it, including precision modes such as TF32, and test correctness on inputs the generator did not see; KernelBench-Verified found shortcuts that assumed positive inputs. Record peak memory as well as time. Report the fraction of workloads that are correct and faster, the size of gains and regressions, and application benefit. Do not average unrelated headline speedups into a single ranking.

5. Design guidelines and forecast

The guidance below is intended for designing a next-generation compiler. It is deliberately performance-first and architecture-flexible. Recommendations should change when better experiments become available.

5.1 Design the system around measurable decisions

5.1.1 Choose the objective before the architecture

For training, minimize representative step time while preserving the required training behavior and resource limits. For serving, maximize throughput under explicit latency and quality constraints, or minimize latency under an explicit load. For edge deployments, startup, memory, and energy may also be hard constraints.

Developer productivity and portability are secondary objectives of similar importance. Record implementation effort and the work needed to reach competitive performance on another target. For a new accelerator, sufficient operator coverage is an enabling condition: a fast isolated kernel cannot run an unsupported application.

Caveat on the objective order. This ordering describes the frontier training and serving segment, where runtime dominates cost and the workload already runs. It is not universal. For new-accelerator bring-up, edge deployments, and the long tail of non-frontier organizations, workload coverage and portability are often the binding constraint and act as the effective primary objective until enough of the application runs to measure performance at all. State the deployment segment when applying this order, and reverse it when coverage or portability, rather than runtime, is what blocks the application. The unresolved question on coverage against peak performance discusses how to allocate work in these cases.

Example: if serving throughput is the objective, a faster matrix multiplication is useful only to the extent that it improves the complete serving path. Profile whether the application is instead limited by communication, memory, scheduling, or another operator. Let those findings select the next experiment.

5.1.2 Expose useful decisions without prescribing a layer count

Decision area	Examples of choices to expose	When to combine it with another area
Workload and graph	Supported shapes, partitioning, fusion, specialization.	When graph boundaries prevent a useful kernel or memory optimization.
Representation and transformation	Layout, memory planning, algebraic rewrites, pass selection.	When local decisions constrain later schedules.
Kernel and machine code	Tiling, pipelining, synchronization, instructions, registers.	When backend behavior determines whether a schedule performs well.
Runtime and distribution	Dispatch, batching, dependencies, placement, communication.	When application bottlenecks cross device or kernel boundaries.
Evaluation and deployment	Validation, measurement, artifact selection, rollout.	When optimization and deployment assumptions differ.

These are a checklist of responsibilities, not five required software layers. Central processing unit (CPU) and accelerator paths can share or separate components as useful. Energy and operating cost can be objectives or constraints; whether they need dedicated representations is an open implementation question.

A shared interface can expose several representations. A unified representation may also be effective. Avoid predicting a fixed count from the number of concerns in today’ s stacks.

5.1.3 Coordinate choices when their performance interacts

Recommended starting point: use profiles and local models to identify promising changes, then measure integrated candidates on the application. Revisit earlier decisions when later choices change their value. Joint search is particularly useful to test when fusion, tiling, communication, and placement interact.

Benefit: the system can find gains that optimizing each component once would miss. **Cost:** the search space and attribution problem grow. **Alternative:** retain local optimizers when their decisions are sufficiently independent and composition performs well. A single central controller is not necessary merely because the application objective is global.

Test: compare independent optimization, coordinated search, and broader synthesis under the same workload and budget. Include a larger-budget comparison to examine attainable performance. The objective is better measured performance; no general proof of a global optimum is claimed. DITRON provides a concrete non-agent comparison for communication and execution decisions; an agent must add value beyond expanding the action space alone.

5.1.4 Decide what to retain, generate, merge, or replace

Architectural choice	Useful starting point	Reason to choose differently	Evidence needed
Existing passes	Reuse mature transformations.	Their action space misses a valuable implementation.	Generated transformations or direct synthesis improve held-out workloads.
Fixed lowering pipeline	Reuse a functioning backend.	It loses important schedule information or cannot express target capabilities.	An alternative path preserves the contract and improves performance or coverage.
Existing kernel language	Use a language with strong tooling and target support.	Representation complexity or hidden scheduling limits search.	Controlled comparison against strong existing languages and interfaces.
Separate agent and compiler	Keep components independently testable.	Compiler and optimizer co-evolution benefits from tighter integration.	Better results with comparable validation and maintenance effort.
One coordinator or several agents	Start with the simplest system that handles the task.	Specialization or parallel exploration yields better results.	Ablations isolate those benefits from extra budget.
Compile-time or offline optimization	Reuse accepted artifacts where workloads are stable.	New shapes or operating conditions justify adaptation.	The performance gain pays for adaptation within deployment constraints.

Replacement is incremental. Generating a new heuristic, replacing a transformation, changing the representation, or bypassing part of lowering all count as architectural change. Retaining an assembler, checker, or fallback does not mean the rest of the compiler has remained conventional.

The next useful controlled comparison places direct PTX synthesis (AI as a Compiler), automatically generated conventional backends (ACT), and compiler-centered agent harnesses (TIRx Harness) against an established compiler. Keep hardware and workload contracts fixed. Measure accepted application runtime, coverage, validation effort, search cost, and maintenance across toolchain changes; a peak kernel speedup cannot settle total engineering value.

5.1.5 Add controller improvement as a separately evaluated loop

Recommended starting point: keep a strong fixed controller and allow offline proposals to change a small, versioned part of it, such as the reusable optimization instructions or retrieval policy. Use ordinary application-search runs to gather development evidence, then evaluate proposed controller versions on separate workloads. Keep the compiler tools fixed for this first comparison. Only accepted controller versions guide subsequent searches.

Within development, the proposer changes a controller, runs that version through complete application-optimization jobs, examines their results, and revises its next proposal. A controller that merely replays a known winning executable has not demonstrated better search. Record controller versions and memory snapshots separately, and measure the accepted application’ s performance as well as the resources spent finding it.

This creates three connected loops: application search produces executables; controller development produces a new optimizer version; compiler evolution produces a new toolchain version. Controller development can use the same agent infrastructure as the other loops. It need not run on the application’ s execution path, modify model weights, or rewrite its own improvement procedure.

Benefit: one controller improvement may help many future workloads. **Tradeoff:** evaluating it requires many complete search jobs, and repeated selection can overfit the development set. Broader controller-code edits become worth testing when restricted changes leave measurable failures. Recursive modification of the improvement procedure is a further experiment, not a prerequisite. Prefer a fixed controller when adaptation fails to transfer or cannot repay development and maintenance cost.

Priority caveat. This is the most expensive loop to evaluate, because its unit of measurement is many complete search jobs on held-out families rather than a single compile. Under the performance-first objective and the guidance to improve search economics last, treat controller self-improvement as a late-stage, gated activity: pursue it only after the application-search and compiler-evolution loops deliver repeated value. The build sequence lists it before final cost reduction for continuity, but its near-term return is usually the lowest of the three loops, so do not schedule it as routine mid-build work.

Validation experiment: compare a strong fixed controller, that controller with persistent memory, a controller with evolved instructions or workflow, and a conventional configuration-search baseline. Give them the same compiler actions, hardware access, feedback, model version, and validation contract. Match per-job resources and separately report all controller-development costs; also show performance-versus-budget curves. Hold out workload families and input shapes, repeat noisy measurements, and count failures. Measure the best accepted application performance and the cost to reach it. A cheaper search at comparable application speed is useful, but it is a different result from a faster executable.

Freeze the proposed controller for the initial held-out evaluation. A separate continual-adaptation experiment may update it between tasks, but must disclose task order and prevent future evaluation information from leaking backward. Keep acceptance rules outside the proposed edits. When both the controller and compiler evolve, first measure each change separately, then measure their combination to detect interactions and attribute gains.

Final-evaluation policy: read-only access is insufficient to prevent overfitting. Keep final tasks, inputs, and detailed results outside proposer access and development traces. Declare a submission limit and a reserved or refreshable workload pool before promotion testing. Even pass/fail promotion decisions reveal information, so count those submissions; once a task or result is disclosed for development, retire it from the unseen pool. Report the evaluation history and use fresh reserved workloads for later generalization claims. Apply the same separation when evaluating compiler changes.

The dominant practical threat is attribution rather than contamination alone: a measured gain can come from a newer model snapshot, toolchain drift, or simply more accumulated compute rather than from the controller change itself. Pin the model and toolchain versions for the duration of the comparison, log total compute and wall-clock for every accepted change, and reject a controller version whose apparent benefit does not reproduce under the same fixed versions and a matched budget.

5.2 A practical sequence for a first implementation

1. **Establish the workload and baseline.** Use an application that matters, a strong compiled or library baseline, and explicit semantics and numerical requirements.
2. **Make experiments dependable.** Add candidate identity, repeatable compilation, validation, timing, and a usable fallback.
3. **Compare optimization methods.** Evaluate conventional search, learned policies where available, and agent proposals with the same feedback and resources.
4. **Expand the action space.** Permit new kernels, cross-operator transformations, or machine-code edits where restricted tuning leaves measurable opportunity.
5. **Coordinate interacting decisions.** Add runtime and communication choices when profiling shows that they matter.
6. **Evolve the compiler selectively.** Turn recurring limitations into proposed analyses, transformations, or backend changes; validate against a separate corpus.
7. **Evaluate controller improvement.** Compare persistent experience, evolved instructions, and workflow changes on fresh optimization jobs before allowing broader self-modification.
8. **Generalize and reduce cost.** Extend shapes and targets, improve developer interfaces, reuse successful work, and reduce search expense.

This sequence is a starting recommendation, not a requirement to solve each stage completely before exploring the next. A new accelerator may need operator coverage before there is a meaningful performance baseline.

5.3 What would weaken or strengthen the thesis?

Hypothesis	Evidence that would weaken it	Evidence that would strengthen it
Adaptive agent reasoning adds optimization value.	Strong conventional search matches or exceeds it across held-out tasks with comparable resources and feedback.	Repeated advantages that remain after budget and action-space controls.
Wider synthesis is worth supporting.	Extra freedom increases failures without improving attainable performance.	Better validated implementations beyond the restricted space.
Coordinated optimization is needed for a workload class.	Local optimizers compose equally well on representative applications.	Integrated search finds repeatable application gains missed by local choices.
Compiler co-evolution improves outcomes.	Generated compiler changes overfit and require more maintenance than their benefit justifies.	New components improve unseen workloads and survive software or hardware updates.
Controller improvement benefits future compiler searches.	Gains disappear on fresh workloads, after accounting for development cost, or against a fixed controller with equivalent memory and resources.	Accepted controller changes improve held-out application results or search efficiency with fixed compiler tools and contracts.

Agent advantage grows with poorly captured decision interactions.	Strong non-agent methods match coupled-task results after controlling headroom, feedback, actions, and budget.	Agent advantage is larger on coupled tasks than local tasks under those controls.
Agentic compilation becomes widely adopted.	Persistent cost, integration, or reliability disadvantages narrow it to specialist uses.	Sustained deployment across independent organizations and workload classes.

For the scope/control hypothesis, review available controlled comparisons on **23 September 2027**. If none meet the controls, record the relationship as untested; do not count a collection of unrelated success stories as confirmation. Specify the tasks, budget, and review date when testing a hypothesis. “Agents never improve” and “agents eventually solve everything” are not useful experimental criteria. A negative result can weaken a near-term prediction without disproving a longer-term possibility.

Controller research checkpoint — review 23 September 2027: look for a public controller-change artifact evaluated on at least two held-out compiler-workload families against a strong fixed controller, a memory-only variant, and a supplied-knowledge variant (the same controller given curated references or expert designs, so that a reference-library gain is not scored as a procedure improvement). Require fixed toolchain and model versions, comparable per-job resources, disclosed development cost, and application-level validation. Record missing evidence as untested, not as proof of impossibility. This is a new research checkpoint; it does not replace the existing horizon predictions. Confidence in useful restricted improvement is medium; timing and architectural form of broad recursive compiler-controller improvement remain low.

5.4 Evidence that changes design confidence

Several examples are particularly informative. Helion distinguishes cheaper tuning from a faster final kernel. CAKE compares different agent programming environments. Ave demonstrates targeted compile-time feedback. AsmEvo makes the final binary an optimization surface. Magellan shows that agent work can be amortized into compiler source. TritorX studies coverage on emerging hardware.

Read their conditions before drawing architectural conclusions. The detailed evidence register records which results have multiple supporting systems and which depend on one study or organizational setting.

Watch the experimental base. Several of these informative results rest on a single organization or a single hardware shape. The CAKE comparison uses three clean-start runs on one accelerator shape, and the Ave results cover selected accelerator families with different denominators for throughput and for validity. Such results are useful for motivating an experiment, but they should stay tentative until independent reproduction across more shapes and vendors. Do not let a single-setting result anchor an architecture decision on its own; require the corroborated-direction claims in the evidence register before committing.

5.5 Forecast over five horizons

All dates are measured from September 2026. Separate confidence in the **direction**, **timing**, and **architectural form**; none is a calibrated probability. The predicates below are this guide’s declared assessment criteria, not thresholds estimated from the literature. They make the calls reviewable without inventing numerical confidence.

A qualifying result identifies the workload, hardware, strong non-agent baseline, numerical and quality requirements, search budget, validation, and failures. A claimed improvement must exceed reported measurement uncertainty. Count independent implementations or organizations, not several reports about one system. Public artifacts or a sufficiently detailed first-party evaluation qualify where specified; a product announcement alone does not.

At each review date, record each predicate as **met** or **not met by available public evidence**, with source versions and a reason. Missing evidence means a miss for the dated public-evidence call, not proof that no private system exists. Reduce timing confidence when a call misses; reduce direction confidence when controlled comparisons contradict its mechanism. Keep the original prediction and record revisions separately.

The last part of Figure 1 summarizes these dated calls; details, qualifying rules, and evidence that would change each call follow below.

5.5.1 One year: 2027

Central forecast: tuning, kernel synthesis, diagnosis, and selected compiler-engineering tasks become more integrated with existing toolchains. Conventional backends remain common because they already implement useful behavior. **Confidence:** direction high; timing medium; architectural form medium.

Observable predicate — review 23 September 2027: at least two independent organizations publish a compiler/serving integration after 23 September 2026, each adding an optimization action or target backend and providing a qualifying application evaluation showing an agent-derived improvement. Additional papers about an unchanged integration do not count.

Evidence that changes the call: fewer than two qualifying integrations misses the timing call; matched evaluations showing no added value from agent decisions lower directional confidence. Two independent reproduced application gains would strengthen it.

Running tally (updated 15 October 2026): 0 of 2. KDA[2] (NVIDIA, 27 September) and AI as a Compiler (Stanford/EPFL, 29 September) are kernel-level results without an application evaluation; the TIRx Harness and KCoral are infrastructure; KernelOPT is a benchmark integration. None qualifies yet. Record candidates here as they appear so the September 2027 review has a trail.

More ambitious outcome: one workflow adapts kernels, configurations, and compiler components together. **Needed progress:** dependable evaluation, useful feedback, and repeatable deployment.

5.5.2 Three years: 2029

Central forecast: leading systems coordinate a wider set of graph, kernel, communication, dispatch, and heuristic decisions. Some generated transformations replace fixed choices. **Confidence:** direction medium-high; timing medium; architectural form low.

Observable predicate — review 23 September 2029: at least one public system jointly searches two decision areas, one involving communication or runtime execution, and demonstrates application gains on two workload families over both independently optimized components and a strong non-agent joint-search baseline. The comparison must hold actions, feedback, contracts, and budgets comparable.

Evidence that changes the call: broader interfaces without a qualifying comparison do not meet the predicate. Non-agent joint search matching the results weakens the claim about agents, while leaving the case for broader scope intact. Transfer to a second accelerator family strengthens it.

More ambitious outcome: an optimization service maintains implementation families with little task-specific intervention. **Needed progress:** transferable hardware knowledge, broader numerical and concurrency checks, and economical reuse.

5.5.3 Five years: 2031

Central forecast: some compilers become continuing optimization services for well-specified workloads, improving both implementations and selected compiler components. **Confidence:** direction medium; timing medium-low; architectural form low.

Observable predicate — review 23 September 2031: at least one publicly inspectable toolchain generates an analysis, transformation, or lowering component and uses it in its default compilation path across two subsequent releases. Evaluation must show maintained application benefit on two held-out workload families under the declared contracts. Merely selecting pass order or changing scalar tuning parameters does not satisfy this component-generation test.

Evidence that changes the call: a component used only on the search workload, or removed because of regressions, does not meet the predicate. Sustained benefit after a hardware or compiler update strengthens the forecast; repeated generalization failures weaken it.

More ambitious outcome: workload-specific compilers are synthesized or substantially restructured. **Needed progress:** specifications, evaluation, and component-generation methods that generalize. Retaining a checker or assembler does not disqualify a result. Direct AI lowering is a precursor, not a qualifying component: it produces one kernel per paid search rather than a reusable lowering used in a default path; ACT’s symbolic backend generation is the conventional comparison arm for this predicate.

5.5.4 Ten years: 2036

Central forecast: some platforms repeatedly generate substantial compiler components and explore algorithms, execution strategies, and hardware together. **Confidence:** direction medium-low; timing low; architectural form low.

Observable predicates — review 23 September 2036: (1) at least two independent toolchains each generate two kinds of component from analysis, transformation, and lowering, with qualifying evaluation across three releases spanning at least six months; (2) at least one public system uses agent proposals to search a hardware design parameter and an execution strategy together, demonstrating a workload benefit against separately optimized baselines on measured hardware or a simulator calibrated against measurements. A hardware design parameter changes a proposed device, such as its memory capacity or execution units; choosing an existing device alone does not count. Record the two outcomes separately. Simulation does not establish fabricated-chip performance.

Evidence that changes the call: isolated demonstrations do not satisfy repeated use; uncalibrated hardware estimates do not satisfy the co-design test. Sustained independent results strengthen the direction. Persistent validation or maintenance failures weaken it even if more components can be generated.

More ambitious outcome: intent-to-implementation systems make familiar pass and lowering boundaries largely unnecessary internally. **Needed progress:** richer specifications, scalable validation, accurate hardware feedback, and reliable search over interacting spaces. If analog or photonic units are included, partitioning and scheduling must preserve statistical quality, drift, and tail-error requirements; the statistical validation contract above supplies the framework rather than assuming deterministic execution.

5.5.5 Beyond ten years

Speculative scenario: a system synthesizes implementations from workload intent and deployment requirements while proposing changes to its software and hardware environment. Conventional compilers may be internal tools, generated components, or alternative routes. **Confidence:** direction low; timing unassigned for the full scenario; architectural form very low.

Observable precursor — review 23 September 2039: at least one public experiment applies the same optimization specification and agent policy to two hardware families, one withheld during system development, and jointly generates program implementations and a compiler component. A qualifying evaluation must compare against existing toolchains and disclose task-specific human interventions. This precursor is a dated test, not a deadline for universal autonomous compilation.

Evidence that changes the scenario: repeated dependence on target-specific hand construction weakens broad transfer. Successful independent reproduction strengthens it. If the precursor is not met, record the miss and revise its review schedule rather than moving the original date silently. One representation or several may remain useful; no date is assigned to complete autonomous chip design.

5.6 Implications for the software stack and hardware co-design

Part of the system	Design opportunity	Practical boundary to examine
Framework and model capture	Preserve useful workload intent, shape information, and profiling context.	Where eager execution, capture, and specialization should meet.
Languages and representations	Expose useful schedule choices and diagnostics.	How much detail to infer, generate, or make explicit.
Compiler implementation	Learn or synthesize heuristics, transformations, and backend support.	Which components can change independently and how regressions are checked.
Runtime and distribution	Optimize dispatch, dependencies, memory movement, and communication.	When a static schedule or persistent kernel beats a general runtime.
Artifact and experiment storage	Reuse kernels, configurations, compiler changes, and failed experiments.	Which assumptions invalidate prior results.
Hardware development	Use simulation and executable workloads to reveal capability and performance gaps.	How compiler findings become tested hardware proposals.

Workload specialization, compiler improvement, validation and review, and hardware enablement/co-design remain useful work categories. Controller improvement cuts across them by changing how the system proposes and coordinates that work. These categories describe responsibilities; they do not prescribe an agent count or a permanent topology.

For new hardware, separate coverage from performance. First determine whether the required application can run under its contract. Then optimize its important paths; some coverage and performance work can proceed together. Simulation can start before silicon is available, but performance claims must account for simulator fidelity. Longer-term co-design can be more ambitious than today’s feedback loop without assuming autonomous chip completion by a fixed date.

5.7 Product and engineering decisions

These are design guidelines, not conclusions established by the cited systems. Choose according to deployment needs and validate the tradeoffs.

Interfaces, memory, and orchestration

Decision	Recommended starting point and reason	Alternative and tradeoff
Agent/compiler interface	Structured inputs and results for compilation, validation, timing, and artifact identity. This makes experiments inspectable.	Flexible source and text interfaces can accelerate exploration but need stronger parsing and validation.
Long-running memory	Keep accepted artifacts and experiment results in durable versioned storage; use summaries and retrieval to help search.	A single growing conversation is simple but harder to reuse and audit.
One agent or specialists	Start with a bounded workflow; add specialists where parallelism or domain separation helps.	A general agent offers flexibility; a hierarchy adds coordination and evaluation cost.
When optimization runs	Reuse accepted artifacts for stable workloads; optimize offline or during a controlled compilation phase.	Frequent adaptation can improve changing workloads but adds latency and operating cost.
Validation strategy	Combine checks matched to the allowed changes and numerical requirements.	Narrow verified actions simplify acceptance; broader synthesis can unlock performance but expands validation work.
Ownership and provenance	Give released artifacts a maintainer and preserve their source, assumptions, and evaluation.	More automatic acceptance scales output but needs adequate checks and recovery.
Portability strategy	Permit target-specific implementations behind compatible workload contracts.	One language can reduce integration work; multiple languages can expose more hardware capability.
Product form	Deliver usable compiler integrations or optimization services with reproducible artifacts.	A compiler flag, internal platform, or managed service has different support and deployment costs.

Evaluation, economics, and maintenance

Decision	Recommended starting point and reason	Alternative and tradeoff
Product evaluation	Combine public workloads with representative private workloads; report regressions and search budgets.	Either set alone can miss important behavior.
Pricing and operating cost	Measure total optimization and maintenance cost against the useful lifetime of an improvement.	Per-job, artifact, subscription, and outcome pricing create different incentives; outcome pricing needs credible attribution.
Multi-tenant deployment	Keep customer workloads and hardware documents within their required access boundaries.	A hosted service simplifies hardware access but requires stronger isolation and data controls.
Model-provider dependence	Separate the model interface from accepted artifacts and rerun evaluations when models change.	A single provider reduces integration effort; specialization or multiple providers can improve control.
Output and data rights	Establish ownership and permitted reuse before using customer artifacts or trajectories for training.	Shared improvement programs need explicit agreements; this is a contractual design topic.
Joint versioning	Record the workload, compiler, hardware, agent policy, validation contract, and artifact versions.	Lockstep releases simplify support but can slow independent updates.
New-hardware cold start	Use simulation, specifications, and compiler feedback to reach required coverage, then application performance.	Expert seed implementations can reduce risk; pure generation tests greater autonomy.
Review capacity	Prioritize valuable changes and measure reviewer effort and escaped defects.	Automatic acceptance is useful for well-checked changes; broad rewrites need stronger evidence.
Reuse of trajectories	Preserve negative results and reusable knowledge with clear rights and compatibility metadata.	Private traces protect information; shared traces improve external comparison and research.

Runtime operation and recovery

Decision	Recommended starting point and reason	Alternative and tradeoff
Interactive latency	Separate quick assistance from long optimization jobs and expose completion expectations.	Strict timeouts improve predictability but can limit attainable performance.
Recovery from bad changes	Keep a known-good artifact, reversible changes, and a tested rollback path.	Restricting edit scope lowers recovery risk but may exclude useful transformations.
Production experimentation	Use replay and controlled application measurements before broader rollout.	Live trials capture interactions but are slower and require operational support.
Proprietary hardware knowledge	Place manuals and retrieval stores where access and residency requirements allow.	Learning only from compiler feedback reduces document dependence but may require more experiments.
Workflow predictability	Give optimization jobs explicit budgets, stopping rules, and inspectable state.	Flexible tool use can explore more broadly; a fixed workflow is easier to diagnose.
Search resources and capability	Track model calls, compilation, device time, and success rates; reduce cost after proving performance value.	Reuse, small specialist models, and generated heuristics can amortize search; none should be assumed to preserve performance without evaluation.

These 23 topics form a product-design checklist whose choices depend on the intended deployment. Token expense and current tool-use failures constrain today’ s deployments. They do not prove that online agentic compilation can never be practical.

5.8 Techniques to develop and experiments to run

5.8.1 Inside the compiler and toolchain

Technique	Why it matters	Next useful experiment
Agent-facing representations and tool interfaces	Expose decisions and actionable feedback.	Compare existing languages, structured interfaces, and a proposed new representation with the same tasks and optimizer.
Validation and fallback machinery	Expand optimization freedom while preserving the contract.	Measure error detection, false rejection, and overhead for each allowed action class.

Reproducible artifacts and replay	Reuse improvements independently of a search session.	Rebuild and remeasure after model, compiler, and hardware updates.
Heuristic and transformation generation	Amortize optimization into the compiler.	Compare generated logic, learned policies, and expert heuristics on held-out applications.
Hardware feedback and compiler evolution	Address recurring unsupported or poorly performing schedules.	Turn a diagnosed limitation into a component change and measure its benefit and regressions.
Direct target-code generation and synthesized backends	Competing routes to reducing hand-coded lowering.	Compare accepted results, operator coverage, semantics, optimization cost, and retargeting against established lowering.

5.8.2 Evaluation, data, and optimizer infrastructure

Technique	Why it matters	Next useful experiment
Application-level evaluation	Establish whether local improvements matter to users.	Attribute kernel, framework, runtime, and communication gains under a fixed workload contract.
Multi-representation data with failures	Train models on what fails and what merely runs slowly.	Test transfer to unseen operators, hardware, and software versions.
Comparable benchmark progression	Separate transformation, kernel, application, and hardware-coverage outcomes.	Publish full success/regression distributions and performance-versus-budget curves.
Artifact provenance and review processes	Maintain generated code after initial optimization.	Measure review effort, escaped defects, and recovery time.
Optimizer workflow and memory	Improve future searches through reusable experience or controller changes.	Separate memory-only adaptation, evolved instructions, workflow search, and recursive changes; compare fresh tasks with equivalent resources and disclose development cost.

5.8.3 Decision checkpoints

Decision	Evidence sufficient to proceed	Reason to revisit
Adopt an agentic optimizer	Repeated application benefit under a declared contract and budget.	Stronger non-agent methods erase the advantage.
Introduce a new representation	Better attainable performance, search efficiency, or validation that matters for the objective.	Similar gains are available through simpler existing interfaces.
Replace a compiler component	The replacement generalizes and has an acceptable regression and maintenance profile.	It overfits a workload or cannot support required targets.
Expand to another accelerator	Required workloads run correctly and reach competitive application performance.	Source portability hides large per-target manual work.
Increase autonomy	Broader actions pass independent evaluation and recovery checks.	Evaluation gaps allow the optimizer to improve its score without improving the intended workload.

5.8.4 What to prioritize first

Build a trustworthy application benchmark, a strong baseline, and a candidate/evaluation interface. Compare restricted tuning with wider synthesis before committing to a new representation. Expand into compiler evolution and hardware co-design when concrete limitations make those experiments valuable. Improve search economics after showing which capabilities are worth making cheaper.

6. Unresolved design questions

These questions preserve the earlier conflict register. Each states competing choices, their implications, and useful evidence for deciding. Outcomes can differ by workload; coexistence is a valid result.

A workload-dependent answer or coexistence is a legitimate conclusion only when the deciding evidence and a review date are stated. Without them, treat the question as open rather than settled, so that “it depends” does not become a way to avoid a testable commitment. This is the cost of the guide’s deliberately cautious framing: recording a defensible position must not slide into declining to take one.

Generated heuristics or learned decision policies?

Magellan generates executable logic; MLGO uses learned policies within LLVM. Generated code can be easy to deploy and inspect; learned policies can encode decisions that are difficult to express as small rules. Compare both on the same tasks, including inference overhead, maintenance, and transfer. Continued investment in both does not require a single winner.

How broadly do reported performance gains transfer?

Production reports establish value in particular settings. Public benchmarks expose failures and regressions across a different distribution. Neither invalidates the other automatically. Seek matched baselines, application measurements, full outcome distributions, and independent reproduction. Keep kernel latency, search time, and throughput distinct.

Restricted actions or wider code synthesis?

Hints and typed actions simplify checking; wider synthesis can discover implementations outside a fixed space. CAKE and Ave motivate better representations and feedback, while direct-rewrite failures identify current weaknesses. Compare the full environment and, where possible, isolate representation, feedback, and model effects. Report speedups at the verification level the deployment requires: AI as a Compiler reports both, and the gap between numerically tested and provably equivalent results (1.37× versus 0.78× for FlashAttention) is itself a measurement of how much freedom the current checkers can afford. Current failures do not establish an advisory-only future.

One kernel language or several?

One language concentrates tooling and training data. Several languages can expose different hardware capabilities and scheduling choices. Triton, Helion, Gluon, TileLang, CuTe DSL, FlyDSL, Pallas, and other interfaces remain relevant candidates. Judge source reuse and competitive per-target performance separately. A dominant training corpus alone does not prove a superior compiler interface.

The independent CUDA Tile evaluation measures both axes. With FlashAttention-2 as the reference, one 60-line causal-attention kernel reached 2.51 times its throughput on B200 but 53% on the RTX PRO 6000. For matrix multiplication, cuTile needed fewer lines than Triton but was slower on every tested Blackwell shape; Triton reached 62–101% of cuBLAS on all three GPUs without source changes, although it was autotuned on each. This is one study with incompletely matched tuning and no Blackwell-specific attention baseline. It still shows that a language can lead on its best-supported target while trailing on another. NVIDIA's Triton backend for CUDA Tile offers a different route: one front end with several target representations. Its documented early limits, including slow tensor-of-pointer access with CUDA 13.1, show that shared source can still require rewriting for performance.

Test: for each candidate language, measure the same kernels on every deployment target under a declared tuning budget, and report the worst-target result alongside the best.

Per-workload optimization or offline compiler improvement?

Per-workload search adapts to specific inputs and deployments. Offline heuristic or transformation generation amortizes effort across many compilations. Compare the value of specialization against its reuse frequency, deployment latency, and maintenance cost. A system may use both without invoking a language model in the execution path.

The relevant reuse count is the number of useful executions before recompilation, workload drift, or hardware changes invalidate the result—not the number of times a search job runs. For high reuse, even costly search may repay itself. For low reuse, prefer cheap proposals, cached artifacts, or conventional tuning unless broader search clears a declared payback and latency threshold. Review that choice when the workload lifetime changes.

Should the controller improve itself, and at what scope?

Memory adaptation, instruction evolution, workflow redesign, and recursive modification are separate choices. The recommended first experiment changes a restricted controller component and keeps the compiler fixed. General coding-agent success supports trying broader edits, but does not settle their value for compiler optimization.

Expand scope only if it improves accepted application performance or search efficiency on fresh tasks after accounting for cost and regressions. Revisit this choice at the September 2027 controller checkpoint. The answer can differ by workload reuse and deployment constraints; record the evidence that makes a fixed or adaptive controller preferable.

How much of the compiler should agents replace?

Using existing compiler tools is a useful near-term strategy. Generating or replacing those tools is also part of the thesis. CAKE's compiler evolution, Magellan's heuristics, and AsmEvo's post-compilation edits make the boundary more flexible than a permanent control-plane/data-plane split suggests. Evaluate replacement component by component; preserving validation does not disqualify architectural change. Direct AI PTX generation and ACT's non-LLM backend generation now provide concrete competing paths, while PTXBench cautions against extrapolating from the best direct-PTX cases. Measure accepted performance, supported workloads, verification coverage, and total cost.

General code review or compiler-specific validation?

General review tools can help with readability and maintenance. Semantic compiler changes also need suitable reproductions, analyses, and workload checks. Archer and llvm-harness provide relevant compiler-specific examples. Measure defects found, false alarms, and reviewer effort on actual compiler changes rather than generic coding benchmarks alone.

Compilers for AI or AI for compilers?

The survey needs both. Better sharding, synchronization, kernel languages, and runtime integration belong in field coverage even without agents. Learned optimization and synthesis belong where they improve these systems or their construction. Keep this distinction explicit so a broad field survey does not become a catalog of only agent papers.

Coverage first or peak performance first?

TritorX studies operator coverage on emerging hardware; peak-performance agents study a different objective. Coverage can enable the first meaningful application benchmark. Performance then remains the priority for this guide. Use application requirements to allocate work, and report coverage, implementation effort, and application speed separately. An independent new-hardware reproduction would strengthen the industry-wide forecast.

Compiler feedback to hardware or autonomous co-design?

Simulation and kernel generation can reveal hardware and compiler limitations today. Wider autonomous exploration of algorithms, compiler components, and hardware is plausible over longer horizons. The existence of a feedback loop does not establish autonomous chip completion, and its absence today does not establish impossibility. Keep the survey grounded in executable workload evidence and distinguish exploration, implementation, and physical production.

7. Evidence and claim register

The record identifiers below preserve links from earlier revisions. They are maintenance aids, not concepts a reader must memorize. **Evidence status and forecast confidence are separate.** Forecast confidence is qualitative and applies to the stated scope.

7.1 Claims with multiple supporting systems

Claim	Evidence assessment	Main qualification
Agents can improve selected optimization tasks.	Corroborated direction across KernelEvolve, GEAK, CAKE, Ave, and other author evaluations.	Different workloads, budgets, baselines, and validation; not independent replication of a common result.
AI can improve compiler decisions or components.	MLGO, Magellan, and CAKE demonstrate different mechanisms.	Learned policies, heuristic synthesis, and compiler evolution should not be conflated.
Hardware-aware interfaces are useful optimization surfaces.	Multiple documented language and compiler systems.	This does not select one representation or demonstrate that each interface helps agents equally.
Application optimization extends beyond isolated kernels.	DITRON, distributed compilation, dynamic execution research, and serving reports.	Broader scope does not establish that agentic control is necessary.
Reusable agent instructions and harnesses can be optimized automatically.	GEPA, AFlow, Meta-Harness, and agent-code evolution studies supply different mechanisms.	Corroborated outside compilers; compiler-specific breadth and recursive improvement have much narrower support.

7.2 Results with narrower support

Source	Author-reported observation	Supported conclusion and limit
Helion tuning	On 33 B200 kernel/shape cases, 9.8 times fewer configurations and 6.7 times less tuning time; language-model/conventional kernel-latency ratio 1.009.	Cheaper search with roughly equal final speed. Conventional refinement closes six of eight gaps exceeding 5%; this is not a 6.7-times kernel speedup.
CAKE	Three clean-start runs per representation on one B200 Flash-KMeans shape, at 80 million tokens: median best speedup 1.144 for CAKE and 0.928 for direct CUDA/PTX, relative to tuned FlashML.	Supports this complete programming/feedback environment in this setting. Ranges overlap; the experiment does not isolate the representation alone.
Ave version 2	Reports 89-99% of expert-library effective throughput on selected MI300X families; valid results within three attempts for 100%/88% of KernelBench Level 1/2 tasks.	Promising targeted feedback. Throughput and validity use different denominators; neither establishes universal expert-level coverage.
TritorX	Generated 481 operators passing their corresponding OpInfo tests.	Concrete coverage assistance in Meta's setting. Passing tests is not a peak-performance result or universal correctness proof.
KernelEvolve	Reports over 60% inference-throughput and over 25% training-throughput improvements on different internal models and hardware.	Production evidence, with private workloads and organization-specific baselines.

Hyperloom	Reports median 1.73-times inference speedup over 16 workloads.	System-level evidence including framework, precision, serving, and kernel changes; do not attribute the entire gain to kernel generation.
KernelBench-Verified	Single-turn kernels from seven models on one H200: best geometric-mean speedup 0.88 against a TF32-enabled baseline with hidden tests, versus 1.43 under the standard protocol; 1.63 at Level 2 in BF16.	Baseline precision and hidden tests change the conclusion. Not an agentic or multi-turn result; same evidence family as KernelBench.
CUDA Tile evaluation	Independent study: one cuTile attention kernel at 2.51 times FlashAttention-2 on B200 and 53% on RTX PRO 6000; cuTile matrix multiplication at 52–79% of cuBLAS.	Kernel throughput only, not application or agent evidence. Tuning is not matched across languages, and no Blackwell-specific attention library is compared.
GEPA kernel experiments	On AMD XDNA2, one evolved prompt raises mean vector utilization from 4.25% to 26.85% with the same GPT-4o refinement agent.	Limited instruction-improvement evidence. Extra search cost and the kernel evaluation protocol matter; no application or broad controller-transfer conclusion.

AI as a Compiler | 0.83–3.34× autotuned Triton for selected direct-PTX GPU kernels. | Kernel performance, with incomplete proof coverage and nontrivial search cost. |
TIRx Harness | 1.33–6.84× across selected Blackwell attention-kernel families. | Integrated system; harness component benefits not isolated. |
KernelOPT | KernelBench Levels 1–3: 1.40× / 1.15× / 1.07× over `torch.compile`, including fallbacks. | Benchmark integration, not production serving A/B. |
Checker mutation study | The official checker misses 16.9% of independently witnessed synthetic mutants. | Checker adequacy on this fault population, not real-world error rates. |
MetaKernelBench | −19% to +29% paired cross-DSL transferred-skill lift. | Same-problem transfer; extra source cost and negative transfer. |
ACT | Six target platforms, 2.3× more Trainium code-generation coverage than reported baseline. | Compiler coverage, not speed, from a non-LLM alternative. |
KCoral | 2.58× more GPU evaluations/minute in reported B200 study; median timing difference from local measurement 0.425%. | Evaluation service throughput, not kernel speed. |
D2K-Bench | Expert design guidance raises frontier-agent speedup from 1.69× to 2.49× (geometric mean, 26 tasks, B200). | Design-discovery gap under one harness and judge; guidance cost excluded. |
PeepholeBench | Agent patches for 40 missed LLVM peephole optimizations are 97–98% valid but profitable on 9–17% of test cases; best agent profitable on 8 of 19 PRs. | Small InstCombine tasks; snapshot of fast-moving agents. |

Argus was revised and renamed Ave on 14 September 2026. Use version 2' s metrics above for current claims. The historical digest filename remains `argus.md` to preserve links.

7.3 Architecture records

Record	Revised claim	Evidence status	Forecast confidence
A1	Agents can use and increasingly change compiler infrastructure.	Corroborated direction; permanent division of responsibility unproven.	High for near-term integration; lower for future boundaries.
A2	Specialization, compiler improvement, validation, and hardware enablement are useful work categories.	Documented examples; taxonomy is a design aid.	No claim that exactly four roles persist.
A3	Kernels, configurations, heuristics, and experiment records become reusable artifacts.	Documented capability in multiple systems.	Medium-high for broader use.
A4	Adoption depends on useful results across the intended workload distribution.	Design recommendation; adoption pathways vary.	Medium for growing selective deployment.
A5	Current failures motivate better feedback and validation.	Author evaluations of current systems.	No permanent capability ceiling inferred.
A6	Responsibilities may use one or several representations.	Multiple approaches exist; a fixed layer count is unproven.	Architecture remains open.
A7	Coordinate interacting decisions and evaluate application benefit.	Design recommendation with related system examples.	Medium for broader scope; one controller not required.
A8	Compiler replacement can proceed component by component.	Early examples; extent and timing remain forecasts.	Medium-low at five years; low at ten years.
A9	The controller is a separate	Narrow kernel prompt evidence;	Medium for restricted

	possible target of improvement.	broader agent redesign and recursive mechanisms mainly adjacent.	improvement; low timing and form confidence for broad recursive compiler-controller evolution.

7.4 Process, stack, and hardware records

Record	Revised claim	Evidence status and qualification
Process P1	Synthesized heuristics and learned policies are viable alternatives.	Both demonstrated; no universal winner established.
Process P2	Supporting several languages and targets may improve portability.	Multiple systems; comparative maintenance and performance remain workload-specific.
Process P3	Compiler-specific tools can improve review.	Early compiler-focused evidence; universal superiority is not established.
S1	Agent/compiler boundaries can evolve.	Corroborated mechanisms; long-term structure is a forecast.
S2	Portability can combine common contracts with target-specific implementations.	Design option, not a requirement to abandon portable representations.
S3	Reusable optimization artifacts change maintenance and deployment.	Documented examples; broader adoption is a forecast.
S4	Agentic bring-up may shorten new-hardware enablement.	Concrete Meta evidence; an industry-wide dependency is unproven.
S5	Compiler and profiler interfaces can provide agent feedback.	Multiple documented interfaces; interface standards remain open.
S6	Distribution and resource constraints may reshape representations.	Current mechanisms exist; no required layer count follows.
S7	Application objectives should guide coordinated optimization.	Design recommendation; benefit of coordination needs workload-specific testing.
S8	More integrated optimization can coexist with generated compiler components.	Conditional forecast; no fixed replacement deadline.
H1	Simulation can support operator generation before silicon.	Early evidence from TritorX; performance depends on simulator fidelity.
H2	Coverage can enable subsequent application-performance work.	Practical strategy; the phases can overlap.
H3	Wider autonomous hardware/software co-design is plausible.	Long-term forecast with low timing confidence; no claim about autonomous chip completion by 2031.

7.5 Evidence to seek next

Prioritize independent new-hardware reproduction, matched agent/non-agent comparisons, performance transfer after compiler or hardware updates, validation of generated compiler components, and controller changes evaluated on fresh application searches. Verify KOPE's full protocol before using its numerical claims. Watch public deployment reports for both generated heuristics and learned policies. Update forecasts when the evidence changes, rather than requiring every new source to support the current architecture.

8. Representative systems and how to use them

This gallery groups systems by the design question they help investigate. Detailed metrics, organizations, publication venues, and limitations belong in the source index. Grouping systems does not imply equivalent evidence or comparable speedups.

Systems	Mechanism or role	Useful design question
TVM, Ansor, TensorIR, FlexTensor	Representations and schedule search.	How far does conventional optimization already go?
Mirage, Prism	Structural and symbolic tensor-program search.	What can non-agent search discover across execution levels?
MLGO, Magellan	Learned decisions and generated executable heuristics.	Should an optimization be learned online, synthesized offline, or implemented manually?
Meta LLM Compiler, Compiler-R1, AwareCompiler, AutoPass	Pass selection and compiler tuning.	When is choosing existing actions enough?
HintPilot, IntOpt, AgentCompile	Hints, optimization intent, and constrained implementation generation.	How much structure improves the search?

ACCLAIM, Reasoning Compiler	Multi-level optimization and model-guided tree search.	What does agent reasoning add to compiler tools and structured search?
LLM-VeriOpt, T-LLM Compiler	Verification-guided transformations.	Which semantic guarantees can be checked for the allowed changes?
mlirAgent, LLM4IR / LaMIR	Tool use and probes of representation understanding.	What information should the optimizer receive, and where does current capability fail?
IRIS-14B, ComPile, IRCoder	Representation translation and training data.	How do source and compiler-representation knowledge transfer?
Generative Compilation	Compiler feedback during partial code generation.	Can earlier feedback reduce failed candidates?
Compiler.next, Compiler 2.0, MOCHA	Research agendas for broader compilation and compiler construction.	Which future claims need concrete experiments?
CompileIQ	Search over compiler controls.	Can tuning existing implementation choices recover useful performance?
Helion, Triton, Gluon, Triton Low-level Language Extensions	Different levels of kernel abstraction and scheduling control.	Which decisions should be inferred or exposed?
TileLang, TIRx, CuTe DSL, FlyDSL, CUDA Tile, ThunderKittens	Hardware-aware programming interfaces.	Which interface expresses the target's important schedules effectively?
Pallas, MaxKernel, JAXBench	Kernel programming, optimization, and evaluation on tensor processing units.	How does optimization transfer beyond a familiar graphics-processor stack?
GEAK, GEAK v4, CuTeGen, KernelLLM	Agentic or learned kernel generation with different environments.	What produces useful validated kernels under realistic budgets?
CAKE, Ave	Representations and structured diagnostic feedback for agents.	Does changing the environment improve attainable performance or search?
KernelEvolve, KForge, AutoKernel, Ascend hierarchical diagnosis	Hardware-aware optimization and diagnosis.	What improves application performance or retargeting on another accelerator?
TritorX, Zomboss	Operator coverage and emerging-hardware compilation.	How much hardware enablement can be automated?
Triton-distributed, DITRON, Shardy	Communication overlap, distributed tiling, and tensor partitioning.	What can broader non-agent compilation achieve, and what does an agent add?
Event Tensor, ForgeMegakernel	Dynamic execution and generated persistent execution strategies.	Can reducing launch and scheduling overhead improve the application?
Hyperloom, VibeServe	Broader serving-stack optimization.	How should gains across configurations, frameworks, and kernels be attributed?
AsmEvo	Optimization of compiled assembly/code objects.	What opportunity remains after conventional lowering?
KernelBench, KernelBench-X, KernelBench-Verified, FlashInfer-Bench, KernelGenBench	Complementary evaluation surfaces.	Which combination measures correctness, performance, transfer, and search cost?
Kernel-headroom study and DLRM-Bench	Profiling and projected application impact.	Which runtime share can the permitted changes reach?
IBM Analog Hardware Acceleration Kit	Repeated evaluation with noise and drift models.	What statistical contract should approximate execution satisfy?
KernelBook, TritonRL, DRTriton, AMDKernelVault	Kernel data generation and model training.	Does better data improve speed as well as correctness?
Archer, llvm-harness, LLVM review experiments	Compiler-aware review and repair.	How much semantic checking and human review remain necessary?
GEPA	Reusable prompt evolution, including preliminary kernel experiments.	Does the improved instruction transfer to fresh application searches?
Automated Design of Agentic Systems, AFlow, Meta-Harness	Automated agent or workflow design.	Which controller edits improve compiler search under comparable resources?
Self-Improving Coding Agent, Darwin Gödel Machine, Hyperagents	Agent-code and improvement-procedure evolution.	Does better coding or agent design transfer to better compiler optimization?
KOPE	Kernel-search memory adaptation; abstract-only review.	Does the full protocol establish transfer beyond repeated-task reuse?
FlowCompile, Auto, AgentFlow, heterogeneous agent serving	Optimizer-workflow compilation, analysis, and placement.	Does this infrastructure help a compiler controller, independently of self-modification?
Agent Skills, SIGIL, SkCC, SkVM, SkillSmith,	Instruction packaging, skill compilation, and	Which adjacent mechanisms improve long-

SKILL.state, DeepSeek Harness	runtime state.	running optimization?
Anthropic Claude C Compiler	Agent-assisted compiler construction.	What do construction experiments reveal about tests, specifications, and maintenance?

Do not confuse similarly named systems: TritorX is Meta's operator-generation work; TIRx is TVM's compiler representation; Triton Low-level Language Extensions extends Triton. ForgeMegakernel and AMD KernelForge are different projects. The SkillSmith compilation paper and the SkillSmith co-evolution paper also have different goals.

9. How to maintain this guide

Select evidence that can change a decision

Search the whole relevant compiler field, including strong non-agent alternatives. Prioritize sources that inform the objective, available actions, representation, validation, deployment, or hardware enablement. Keep generic coding-agent and chip-design material adjacent unless it connects to a compiler decision.

Walk primary papers, official documentation, repositories, and first-party engineering reports. Check mature stacks for recent changes; do not search only titles containing "language model" and "compiler." If a language or tool is central to a claim, summarize the language or tool itself as well as the agent that uses it.

Update in a coherent order

1. Read the current source, including its version, date, method, and evaluation conditions.
2. Add or update the source digest with organization, publisher, relevance tier, evidence type, and limitations.
3. Update the publication index and the product or repository map when relevant.
4. Revise affected design guidance, unresolved questions, and evidence records together.
5. Check the entire narrative for contradictory objectives, architecture assumptions, and timelines. If components, improvement loops, evaluation boundaries, or the forecast schedule changed, re-render the blueprint figure with `python3 scripts/build_blueprint.py` and revise its caption.
6. Rebuild and inspect the survey PDF after the narrative is settled. Update slides only when requested or when their content is explicitly being revised; keep both transcript languages synchronized with any slide edit.
7. Record the change and run repository validation. Commit on the existing main branch and publish only when authorized; an explicit no-push instruction takes precedence.

Write for a designer reading from beginning to end

Lead with the decision and explain its purpose. For substantial guidance, state the recommended starting point, benefit, tradeoff, conditions for choosing differently, and the experiment that would settle the choice. Use ordinary technical language and concrete examples.

Expand necessary abbreviations at first use. Keep established system names, but avoid strings of unexplained acronyms. Use descriptive link text and brief local explanations instead of symbol-heavy section references. Keep old record identifiers in the evidence register or compatibility anchors, not as the language of the argument. Prefer compact tables to dense cross-reference matrices.

Keep the evidence honest

Distinguish a documented capability, an author result, independent corroboration, a recommendation, and a forecast. For numerical claims, state the baseline, measured object, target, workload, budget when known, and validation scope. Count related publications as one evidence family. Never describe a correctness-only result as a speedup or a kernel speedup as an application gain without application measurements. For self-improvement claims, identify the changed artifact: application, persistent memory, controller instruction/workflow, compiler component, or the improvement procedure itself. A feedback loop or an architecture diagram alone establishes none of these improvements.

Treat failures as current observations with possible solution paths. Describe what evidence would change the prediction. Never rewrite inconvenient evidence merely to preserve the agentic thesis or today's compiler architecture. Give every forecast a review date, an observable predicate, and an evidence rule; retain the original call when recording its outcome.

Known coverage work

Deepen less mature source summaries before using them to change forecasts. Extend coverage of CPU, mobile, embedded, and distributed workloads. Track independent bring-up evidence and stronger controlled comparisons. Existing watchlists include Mosaic GPU, Mojo, HipKittens, and hardware-specific agent systems, plus adjacent workflow and skill-runtime research when it contributes to an executable compiler loop.

Reference guide

This directory stores the evidence behind the survey and design guide. The narrative explains the design choices; these sources let readers examine their basis.

Collection	What to use it for
Publication index	Find a digest and its primary source, organization, publisher, and date.
Product evidence	Understand reported integrations, commercial offerings, and research prototypes.
Implementation evidence	Find code, evaluation tools, and compiler interfaces for experiments.

Reading evidence

A paper, blog, repository, and talk about one system are one evidence family. Several systems can support a direction without independently reproducing the same result. Related research such as Mirage and Prism, or DITRON and Triton-distributed, also shares a lineage.

Relevance tiers describe how a source informs a design decision: direct evidence, supporting infrastructure, or background. They are not quality ratings. A star in the index indicates reading priority. Read the evaluation conditions and limitations before using a numerical result.

Distinguish runtime speed from tuning time, kernel gains from application gains, and workload coverage from peak performance. Record the hardware, baseline, budget, source version, and validation scope. Label projected application gains separately from measured gains; a runtime profile does not automatically transfer to another workload or organization. The current Ave digest retains its historical `argus.md` filename for compatibility.

Controller-improvement sources distinguish reusable instructions, persistent experience, workflow code, and recursive changes to the improvement procedure. Begin with GEPA for limited kernel-specific evidence, then Meta-Harness and Hyperagents for broader mechanisms. KOPE remains an abstract-only lead. General agent benchmarks do not establish compiler-performance gains.

Adding or refining a source

1. Use the digest template, including organization, publisher, primary link, and limitations.
2. Update the publication index and its total. Keep related sources visibly connected.
3. Explain which design decision changes, or why the evidence leaves the choice unresolved.
4. Update product and implementation maps when the source adds a useful interface or deployment fact.
5. Follow the repository Survey skill, run `python3 scripts/validate_survey.py`, and rebuild the PDF after narrative edits.

Use descriptive links and plain language. Detailed evidence belongs here; the main guide should remain readable from beginning to end.

Products (prediction signals)

These offerings and first-party reports inform the design guide. Research prototypes are included when they expose useful implementation choices; their inclusion does not imply a supported commercial product. See also the implementation map.

Optimization services and integrations

Offering	Reported role	Design implication or limitation
AlphaEvolve on Google Cloud	Evolutionary optimization offered as a service.	A deployment model for expensive search; service availability does not establish compiler superiority.
Google Magellan and MLGO	Generated compiler heuristics and learned decision policies.	Compare both ways to amortize optimization into a compiler.
NVIDIA CompileIQ and agent skills	Workload-specific compiler configuration and optimization workflows.	Preserve reusable configuration artifacts and validate actual application benefit.
AMD GEAK	Kernel optimization and application profiling/measurement workflows.	Let application bottlenecks select kernel work; check the version and protocol.
AMD Hyperloom	Application optimization with kernel work delegated to GEAK or KernelForge.	Evaluate the complete serving path and separate framework from kernel gains.
NVIDIA TensorRT-LLM agent integration proposal	Compiler and kernel workflows inside a serving stack.	A proposed integration is a signal, not evidence of universal default deployment.
Helion and its model-guided tuner	Higher-level kernel programming and automated configuration search.	Measure tuning time separately from final kernel latency.
FlashInfer-Bench	Evaluation and substitution of kernels in serving workloads.	Connect a kernel experiment to application measurements.

Research systems that affect product design

System	Useful capability	Scope of evidence
MaxKernel, Pallas, JAXBench	Kernel optimization and evaluation on tensor processing units.	Performance on a supported platform; not proof of coverage on a new chip.
Meta LLM Compiler and KernelLLM	Models specialized for compiler or kernel tasks.	Model releases are ingredients for a system, not complete deployment evidence.
Meta TritorX and KernelEvolve	Hardware enablement and performance optimization across accelerator targets.	Separate coverage results, kernel measurements, and industry deployment reports.
GEPA	Evolved instructions for fixed-model agents, with preliminary kernel experiments.	Research evidence for a restricted controller change; no production self-evolving compiler claim.
Meta-Harness and Hyperagents	Automated harness design and editable improvement procedures.	General-agent research prototypes; compiler transfer remains an experiment.
CAKE	Schedule representation and compiler feedback developed with agents.	Author evaluation; compares complete programming environments.
Ave, formerly Argus	Compile-time data-flow assertions and useful failure feedback.	Revised September 2026 MI300X results; do not reuse older Argus metrics.
CUDA Tile and its Triton backend	Tile-oriented programming and compiler representation.	An independent evaluation finds strongly target-dependent attention performance; neither establishes the best agent architecture.

Existing compiler and runtime choices

These are baselines and possible components. Their maturity is a reason to evaluate reuse, not a permanent requirement to preserve their boundaries.

Platform	Role in a design experiment
NVIDIA TensorRT-LLM and CUDA libraries	Established serving and kernel baselines.
FlashInfer	Attention and related kernels used by serving systems.
PyTorch `torch.compile` and Inductor	Application capture, compilation, and integration.
DITRON / Triton-distributed	A non-agent distributed compiler; evaluate

	computation/communication overlap and application integration.
OpenXLA XLA, StableHLO, and Shardy	Model representation, compilation, and distributed tensor partitioning.
JAX Pallas and Mosaic	Kernel programming and compilation on supported accelerators.
Modular MAX and Mojo	An alternative compiler and runtime stack.
Intel OpenVINO	Deployment toolkit for supported processors and devices.
AWS Neuron and its kernel interface	Compilation and kernel control for AWS accelerators.
Qualcomm AI Hub, Qualcomm Neural Network tools, and Hexagon-MLIR	Device deployment and an open compiler path for Hexagon.

Adjacent infrastructure and historical context

Item	Why it is included cautiously
Olive, ONNX Runtime, Hugging Face Optimum	Deployment and packaging context; assess separately from agent optimization.
OctoML	Historical commercial context for automated tuning; avoid assuming a current offering.
Generic AI code-review products	Workflow evidence until connected to compiler-specific checks.
Anthropic’ s C compiler experiment	Evidence about agents constructing compiler software; not a general commercial compiler result.
DeepSeek Harness	General agent runtime; compiler-specific evaluation must be added.
Agent Skills specification	A packaging format, not a compiler optimization method.
SIGIL, SkCC, SkVM, SkillSmith, and SKILL.state	Skill compilation and workflow-state research; compiler-task transfer needs measurement.

Update this map when a source changes an interface, deployment fact, or design decision. A paper, product page, and code release about one system remain one evidence family.

Repos & forge evidence

Use these projects to investigate interfaces, baselines, and experiments for the design guide. A repository shows inspectable artifacts; it does not by itself establish measured benefit, current maintenance, or independent reproduction. Check the relevant source digest and version before relying on it.

Optimization and compiler engineering

Project	What to inspect	Question to test
Archer and llvm-harness	Review and repair workflows using compiler-specific tools.	Does feedback improve defect detection and reviewer effort?
OpenEvolve and HeuriGym	Evolutionary search and heuristic evaluation.	Do generated heuristics generalize to held-out applications?
ACCLAIM	Cooperation between a language model and compiler at several levels.	Which interactions contribute measurable gains?
mlirAgent	Compiler tools and experiments with representation edits.	Which feedback or action-space changes address observed failures?
LLM4IR	Evaluation of models' understanding of compiler representations.	Does better understanding translate into better optimization?
Compiler-R1	Reinforcement learning with compiler tools.	How does learned tool use compare with conventional search?
HintPilot	Restricted compiler hints and pragmas.	What performance is lost or gained by restricting actions?
CompileIQ	Compiler configuration search and reusable artifacts.	Does the selected configuration improve the intended workload?
Claude's C compiler	An agent-built compiler implementation.	How much workload and target coverage survives independent testing?
MLGO training tools	Learned compiler policies.	When is a learned policy preferable to generated heuristic code?
CompilerGym	Compiler optimization environments.	Are the action space and reward representative of deployment?

Magellan's papers and LLVM presentation provide the research account. OpenEvolve and HeuriGym offer related public experimentation routes; they should not be represented as independent reproductions of Magellan's production results.

Kernel and application optimization

Project	What to inspect	Evaluation boundary
KernelBench and KernelBench-X digest	Kernel generation, correctness, and performance protocols.	Passing a test does not establish a speedup or full numerical coverage.
KernelAgent	Generation and validation of Triton kernels.	Separate kernel outcomes from application outcomes.
GEAK and GEAK-agent	AMD kernel workflows and application-level evaluation.	Match versions, workloads, and baseline configurations.
Hyperloom	Serving optimization and delegation to kernel tools.	Attribute runtime/framework gains separately from kernel gains.
Accelerator agents	MaxKernel and JAXBench on supported targets.	Target-specific peak results do not establish new-chip coverage.
KernelGenBench	Tasks across sources and devices, including search-cost measurements.	Compare success and regression distributions under declared budgets.
AutoKernel	Profiling-directed application and kernel work.	Measure whether optimizing the selected bottleneck helps the application.
FlashInfer-Bench	Workload traces, kernel evaluation, and serving integration.	Keep kernel validation and application validation distinct.
VibeServe	Research on serving-stack synthesis.	Evaluate the whole generated execution path and deployment assumptions.
Helion	Tile programming, configuration spaces, and tuning.	Consult the 2026 tuning report for model-guided search evidence.
Kernel-headroom artifacts	Workload profiles, numerical checks, and	Inspect the classification and projection

	DLRM-Bench.	assumptions before generalizing.
KernelBench-Verified	TF32 baseline, hidden input transformations, and peak-memory measurement for KernelBench.	Fixed hidden transformations can be anticipated; keep the baseline precision explicit.
CUDA Tile evaluation artifacts	Matrix-multiplication and attention kernels across cuTile, Triton, and CUDA baselines.	Check tuning parity and baseline versions before comparing languages.
Triton AI Compiler and PTX Gym	Direct PTX candidates, checking and optimization traces.	Does direct synthesis beat Triton after counting all failed trials and GPU time?
TIRx Harness	Compiler analysis, references, experiments and candidate frontiers.	Which harness components make an independent contribution?
Kernel generation bibliography	Discovery of further primary sources.	A bibliography is not an evaluation.

TritorX and KernelEvolve have important paper and industry-report evidence. Do not assume full public implementations or reproducibility merely because related libraries are public.

Compiler interfaces and reusable components

Project	Why it belongs in an experiment
LLVM	Existing analyses, transformations, code generation, and learned-policy integration.
ACT backend generation	Non-LLM synthesis of compiler backends from accelerator instructions; compare coverage, performance and engineering costs.
TVM and TIRx kernels	Tensor representations, schedule controls, and example kernels.
TileLang and language tools	Tile programming, layout inference, and editor/tool feedback.
Triton, Gluon digest, and Triton extensions	Different levels of layout and scheduling control.
Triton-distributed, also the DITRON project	Related implementations of joint computation, memory movement, and communication; one research lineage.
FlyDSL	Python-based control for AMD kernel programming.
CUTLASS and CuTe	NVIDIA kernel components and layout/scheduling interfaces.
ThunderKittens	Tile abstractions embedded in CUDA.
OpenXLA, StableHLO, and Shardy	Model compilation and distributed tensor decisions.
PyTorch `torch.compile` and Inductor	Application-facing compilation and integration.
TensorRT-LLM and FlashInfer	Serving execution and kernel integration targets.
KernelBook	Paired PyTorch and Triton training examples; inspect coverage and leakage.
Mirage and Prism	Structured superoptimization baselines and possible search components.

IBM's analog hardware toolkit supplies executable noise and drift models for statistical evaluation; simulation and measured hardware evidence must remain distinct.

Controller improvement experiments

Project	What to inspect	Evaluation boundary
GEPA	Reflective optimization of reusable instructions.	Reproduce kernel results with strong baselines and fresh workloads.
Automated Design of Agentic Systems and AFlow	Agent-code and workflow search.	Generated task agents do not imply an evolving outer optimizer.
MetaKernelBench	Same-problem cross-DSL skill-transfer evaluation.	Which reused instructions help, and how often does negative transfer occur?
Meta-Harness	Harness-code search using prior source and execution traces.	General-task evidence; compiler adaptation requires measurement.
Hyperagents	Editable task and modification agents.	Some outer selection and evaluation components remain fixed.

These are experiment starting points, not demonstrations that a complete compiler controller evolves autonomously. See the controller source summaries for versions, evaluation scope, and research-family dependencies.

Workflow infrastructure

Project	Scope
KCoral	Isolated shared GPU evaluation and remote diagnostics.
DeepSeek Harness	General agent runtime and session state.
Agent Skills	Skill packaging and discovery.
SIGIL	Skill compilation into executable workflows.
SkCC	Skill representations and code generation.
SkillSmith	Research on skill boundaries and compilation.
Gerrit AI review, ReviewAI, and GerritForge provider	General review interfaces; add and evaluate compiler checks before claiming compiler-specific benefit.

Infrastructure can support a future optimizer without proving that optimizer’ s performance. Negative results are useful evidence for choosing the next experiment; they do not establish permanent limits on the architecture. Detailed accounts are in the publication index.

Publications index

Each title opens its primary source. The complete source digests remain in the repository. Multiple records about one system are one evidence family; the catalog is not a count of independent confirmations.

Surveys & vision

Date and type	Source	Organization and publisher
2026; paper	The New Compiler Stack: A Survey on the Synergy of LLMs and Compilers	ICT, CAS · UCAS · University of Leeds. Published by arXiv.
2025; paper	Compiler.next: A Search-Based Compiler to Power the AI-Native Future of Software Engineering	Queen's University. Published by arXiv.
2026; paper	Reading AI Model Compilation in MLIR Through the Lens of Formal Theories	Qualcomm Technologies International. Published by arXiv.
2026; talk	Compiler 2.0: Building the Next Generation Compilers with ML (Ken Kennedy Award plenary, HPCA/CGO/PPoPP/CC 2026)	MIT CSAIL. Published by ACM/IEEE Ken Kennedy Award · HPCA/CGO/PPoPP/CC 2026.
2022; talk	CGO 2022 Keynote: Compiler 2.0	MIT CSAIL. Published by CGO 2022 · YouTube.
2020; talk	Compiler 2.0: Using ML to Modernize Compiler Technology (CC 2020)	MIT CSAIL. Published by ACM CC 2020.
2025–28; company	DARPA MOCHA / Aarno Labs: Compiler 2.0 project	Aarno Labs · MIT · UIUC (DARPA MOCHA). Published by Aarno Labs / DARPA.
2026; paper	Towards Automated Kernel Generation in the Era of LLMs	BAAI · PKU · HKUST(GZ) et al.. Published by arXiv.

Classic DL compilers

Date and type	Source	Organization and publisher
2018; paper	TVM: An Automated End-to-End Optimizing Compiler for Deep Learning	UW · AWS · Berkeley et al.. Published by USENIX OSDI 2018.
2020; paper	Ansor: Generating High-Performance Tensor Programs for Deep Learning	UW · AWS · OctoML et al.. Published by USENIX OSDI 2020.
2021; company	TVM blog: Introducing Auto-scheduler (Ansor)	Apache TVM. Published by Apache TVM blog.
2020; paper	FlexTensor: An Automatic Schedule Exploration and Optimization Framework	Peking University et al.. Published by ACM ASPLOS 2020.
2022; paper	TensorIR: An Abstraction for Automatic Tensorized Program Optimization	UW · AWS · OctoML et al.. Published by ACM ASPLOS 2023.
2026; company	TIRx: An Open Compiler Stack for Evolving Frontier ML Kernels	Apache TVM · MLC (CMU course). Published by Apache TVM blog.
2025/26; paper	TileLang: A Composable Tiled Programming Model for AI Systems	Peking University · Microsoft Research · Imperial College London. Published by arXiv.
2021; paper	MLIR: A Compiler Infrastructure for the End of Moore's Law	Google · multi-institution. Published by arXiv (MLIR).
2025; company	OpenXLA StableHLO roadmap	OpenXLA / Google. Published by OpenXLA.
2025; company	Helion: High-Level DSL for Portable ML Kernels	Meta (PyTorch). Published by PyTorch blog.
2026; paper	Hexagon-MLIR: AI Compilation Stack for Hexagon NPUs	Qualcomm. Published by arXiv.
2025+; code	pytorch/helion (GitHub)	Meta (PyTorch). Published by GitHub.
2024+; company	Pallas: JAX kernel language for GPUs and TPUs	Google (JAX). Published by JAX documentation.
2026; company	From Minutes to Seconds: LLM-Guided Autotuning for Helion Kernels	Meta (PyTorch/Helion team). Published by PyTorch blog.
2025/26; paper	Automatically Generating ML Compiler Backends from Tensor Accelerator ISA Descriptions (ACT)	University of Illinois Urbana-Champaign. Published by ACM OOPSLA 2026 · arXiv.

MLGO & RL gyms

Date and type	Source	Organization and publisher
2021; paper	MLGO: a Machine Learning Guided Compiler Optimizations Framework	Google · CMU. Published by arXiv.
2022; company	Google Research blog: MLGO	Google Research. Published by Google Research blog.
2022; company	InfoQ: MLGO Framework Brings Machine Learning in Compiler Optimizations	InfoQ (covering Google MLGO). Published by InfoQ.
2022+; code	LLVM docs: Machine Learning Guided Optimization (MLGO)	LLVM / Google MLGO. Published by LLVM docs.
2022; code	CompilerGym (Meta/FAIR)	Meta FAIR. Published by GitHub.
2022; forum	LLVM Discourse: ML-guided ordering of compiler optimization passes (GSoC)	LLVM community (GSoC). Published by LLVM Discourse.

Foundation LLMs for compilers

Date and type	Source	Organization and publisher
2023; paper	Large Language Models for Compiler Optimization	Meta. Published by arXiv.
2024; paper	Meta Large Language Model Compiler: Foundation Models of Compiler Optimization	Meta. Published by arXiv.
2024; company	Meta AI publication page: LLM Compiler	Meta AI. Published by Meta AI Research.
2024; company	Chris Cummins: LLM Compiler foundation models post	Meta (Chris Cummins). Published by LinkedIn.
2024; paper	Compiler generated feedback for Large Language Models	Meta. Published by arXiv.
2023; forum	HN: Large Language Models for Compiler Optimization	Hacker News community. Published by Hacker News.
2024; forum	HN: Meta LLM Compiler (long thread)	Hacker News community. Published by Hacker News.
2024; forum	HN: Meta Large Language Model Compiler	Hacker News community. Published by Hacker News.
2023; paper	ComPile: A Large IR Dataset from Production Sources	UC Davis • TUM • LLNL • UIUC • ANL • Google et al.. Published by arXiv • Hugging Face.
2024; paper	IRCoder: Intermediate Representations Make Language Models Robust Multilingual Code Generators	TU Darmstadt • University of Würzburg. Published by ACL 2024 • arXiv.
2025; paper	Can Large Language Models Understand Intermediate Representations in Compilers?	Kent State • HUST • PNNL • Chongqing University. Published by ICML 2025 • PMLR • arXiv.
2026; paper	LLM Translation of Compiler Intermediate Representation	Barcelona Supercomputing Center • UPC. Published by arXiv.

Agentic & RL compilers

Date and type	Source	Organization and publisher
2025; paper	Compiler-R1: Towards Agentic Compiler Auto-tuning with Reinforcement Learning	ISCAS • UCAS. Published by arXiv.
2025; code	Mind4Compiler/Compiler-R1 (code)	ISCAS / Mind4Compiler. Published by GitHub.
2026; paper	LLM-VeriOpt: Verification-Guided RL for LLM-Based Compiler Optimization	multi-institution. Published by CGO 2026.
2026; paper	Magellan: Autonomous Discovery of Novel Compiler Optimization Heuristics with AlphaEvolve	Google DeepMind / Google. Published by arXiv.
2025; talk	LLVM Developers' Meeting slides: Magellan	Google DeepMind / Google. Published by LLVM Developers' Meeting 2025.
2025; paper	AlphaEvolve: A coding agent for scientific and algorithmic discovery	Google DeepMind. Published by arXiv.
2026; company	DeepMind blog: AlphaEvolve impact	Google DeepMind. Published by DeepMind blog.
2025; paper	AwareCompiler: Agentic Context-Aware Compiler Optimization	ISCAS • UCAS • NTU Singapore. Published by arXiv.
2026; paper	AutoPass: Evidence-Guided LLM Agents for Compiler Performance Tuning	Shaanxi Normal University • Northwest University • University of Leeds. Published by arXiv.
2026; paper	Beyond Pass-by-Pass Optimization: Intent-Driven IR Optimization with Large Language Models	SKLP, ICT, CAS • UCAS • Jiangnan University • CUHK. Published by arXiv.
2026; paper	HintPilot: LLM-based Compiler Hint Synthesis for Code Optimization	Zhejiang University • Purdue. Published by arXiv.
2026; paper	Agentic Code Optimization via Compiler-LLM Cooperation (ACCLAIM)	AWS AI • Georgia Tech. Published by arXiv.
2026; paper	T-LLM Compiler: Trusted LLM-based Code Optimization and Verification Framework	Huawei Technologies, Heterogeneous Compiler Lab. Published by arXiv.
2026; code	amazon-science/acclaim (GitHub)	Amazon Science / AWS AI. Published by GitHub.
2026; paper	Generative Compilation: On-the-Fly Compiler Feedback as AI Generates Code	ETH Zurich • INSAIT/Sofia University • UC Berkeley. Published by arXiv.
2026; paper	AI as a Compiler: Compiling Triton kernels without the Triton compiler	Stanford University • EPFL. Published by arXiv.
2026; paper	Verified Learning for Compiler Optimization: An LLM-Guided Architecture with Formal Control	Pennsylvania State University. Published by arXiv.
2026; paper	Can Coding Agents Implement Missed Compiler Optimizations? Evaluating LLM Agents on LLVM Peephole Optimizations	University of Waterloo. Published by arXiv.

GPU kernels & inference compilers

Date and type	Source	Organization and publisher
2025; paper	KernelBench: Can LLMs Write Efficient GPU Kernels?	Stanford • Princeton. Published by arXiv.
2025; company	Stanford blog: KernelBench	Stanford (Scaling Intelligence). Published by Stanford blog.
2025; code	ScalingIntelligence/KernelBench	Stanford (Scaling Intelligence). Published by GitHub.

2026; paper	KernelBench-X: A Comprehensive Benchmark for Evaluating LLM-Generated GPU Kernels	Tsinghua University. Published by arXiv.
2025; paper	GEAK: Introducing Triton Kernel AI Agent & Evaluation Benchmarks	AMD. Published by arXiv.
2025; company	AMD ROCm blog: Triton kernel AI / GEAK	AMD. Published by AMD ROCm blog.
2025; code	AMD-AGI/GEAK agent	AMD AGI. Published by GitHub.
2025; company	Meta KernelLLM (Hugging Face)	Meta. Published by Hugging Face.
2025; company	GPUMODE/KernelBook dataset	GPU MODE. Published by Hugging Face.
2025; paper	TritonRL: Training LLMs to Think and Code Triton Without Cheating	Amazon • multi-institution. Published by arXiv.
2026; paper	DRTriton: Large-Scale Synthetic Data Driven RL for Triton	Texas A&M • Oracle. Published by arXiv.
2026; paper	FlashInfer-Bench: Virtuous Cycle for AI-driven LLM Systems	UW • CMU • NVIDIA • UC Berkeley. Published by MLSys 2026.
2026; code	flashinfer-ai/flashinfer-bench	FlashInfer / NVIDIA • UW • CMU. Published by GitHub.
2025; paper	Reasoning Compiler: LLM-Guided Optimizations for Efficient Model Serving	UC San Diego. Published by arXiv.
2026; paper	AgentCompile: An LLM-Guided Compiler for Direct CUDA Inference	City University of Hong Kong. Published by arXiv.
2025; code	ucb-bar/mlirAgent	UC Berkeley (ucb-bar). Published by GitHub.
2025+; code	flagos-ai/awesome-LLM-driven-kernel-generation	FlagOpen / flagos-ai. Published by GitHub.
2026; paper	AutoKernel: Autonomous GPU Kernel Optimization	RightNow AI. Published by arXiv.
2026; code	RightNow-AI/autokernel (GitHub)	RightNow AI. Published by GitHub.
2026; paper	Kernel Forge: Agent Harness for CUDA Kernel Opt	University of Michigan. Published by arXiv.
2026; paper	CuTeGen: Agentic GPU Kernels using CuTe	University of Toronto • Standard Kernel. Published by arXiv.
2026; paper	KernelBlaster: Memory-Augmented In-Context RL for CUDA	NVIDIA • UC Berkeley. Published by arXiv.
2026; paper	CAKE: Compiler–Agent Co-Design for Frontier Kernel Evolution	NVIDIA • Carnegie Mellon University. Published by arXiv.
2026; paper	Ave: Guiding Agentic GPU Optimization Using Data-Flow Invariants	CausalFlow • HKUST • Tsinghua University • Stanford • UCAS • UC Riverside. Published by arXiv.
2025/26; company	Gluon: Triton’s lower-level GPU programming model	Triton project (OpenAI / community). Published by Triton docs.
2026; paper	TLX: Hardware-Native, Evolvable MIMW GPU Compiler for Large-scale Production Environments	UC San Diego • Meta. Published by arXiv.
2026; company	FlyDSL: Expert GPU Kernel Development with the Ease of MLIR Python Native DSL on AMD GPUs	AMD. Published by AMD ROCm blog.
2025; company	Achieve CUTLASS C++ Performance with Python APIs Using CuTe DSL	NVIDIA. Published by NVIDIA Developer blog.
2026; paper	Event Tensor: A Unified Abstraction for Compiling Dynamic Megakernel	Carnegie Mellon University • NVIDIA et al.. Published by MLSys 2026 • arXiv.
2026; company	ThunderKittens 2.0: Even Faster Kernels for Your GPUs	Stanford Hazy Research. Published by Hazy Research blog.
2026; paper	MaxKernel: Agentic Kernel Generation for TPUs	Google • Google DeepMind. Published by arXiv.
2026; paper	JAXBench: Benchmarking Autonomous TPU Kernel Optimization	Google • Google DeepMind • Harvard University • UC Berkeley. Published by arXiv.
2026; paper	AMDKernelVault: Large-Scale Datasets and Agentic Training for AMD GPU Kernel Optimization	AMD. Published by arXiv.
2026; paper	KernelGenBench: Can LLMs and Agents Write Efficient Kernels Across Operator Sources and Hardware Platforms?	BAAI • Beijing Normal University • Beijing Jiaotong University • Institute of Automation, CAS • Peking University. Published by arXiv.
2026; paper	AsmEvo: Agentic Assembly-Level Optimization of AMD GPU Kernels with Functional Equivalence Verification	AMD. Published by arXiv.
2026; paper	ForgeMegakernel: A General Framework for Efficient Auto-Regressive Model Decode Megakernels	Tsinghua University • University of Chinese Academy of Sciences • ModelBest. Published by arXiv.
2026; paper	How Much of a Real Workload Can LLM-Generated GPU Kernels Actually Reach?	Gaurav Agarwal • Ashish Garg • Isha Singhal (affiliations not stated). Published by arXiv.
2026; paper	Evaluating CUDA Tile for AI Workloads on Hopper and Blackwell GPUs	University of Wisconsin–Milwaukee • Illinois Institute of Technology. Published by arXiv.
2026; paper	KernelBench-Verified: Do LLM-Generated Kernels Actually Beat PyTorch?	Meta • Stanford University. Published by arXiv.
2026; company	TIRx Harness: An Open Compiler Harness for Agentic GPU Programming	MLC community (CMU • Apache TVM) • NVIDIA KDA team. Published by MLC blog • GitHub.
2026; company	KDA^2: Kernel Design Agents (KDA) optimize Kimi Delta Attention (KDA)	NVIDIA (Agentic CUDA Effort) • CMU/MLC. Published by NVLabs project blog.

2026; paper	KernelOPT: Dispatch-Aware Agentic Search for GPU Kernel Optimization	Red Hat (PyTorch team). Published by arXiv.
2026; paper	KernelZero: Co-Evolving Proposer and Coder for Continuously Improved GPU Kernel Generation	Institute of Computing Technology, CAS · UCAS · Institute of Software, CAS. Published by arXiv.
2026; paper	PTXBench: Benchmarking and Adapting LLMs for GPU Kernel Optimization with Architecture-specific PTX	Stanford University · Carnegie Mellon University · RadixArk. Published by arXiv.
2026; paper	D2K-Bench: Can LLM Agents Turn Expert Designs into Efficient GPU Kernels?	Alibaba Group · HKUST · USTC. Published by arXiv.

Company infra

Date and type	Source	Organization and publisher
2025; company	NVIDIA: Focus on Your Algorithm—CUDA Tile Handles the Hardware	NVIDIA. Published by NVIDIA Developer blog.
2026; company	NVIDIA: Develop High-Performance GPU Kernels in C++ with CUDA Tile	NVIDIA. Published by NVIDIA Developer blog.
2026; company	NVIDIA CUDA 13.3: Tile C++ + CompileIQ	NVIDIA. Published by NVIDIA Developer blog.
2026; company	NVIDIA: Extract More Kernel Performance with CompileIQ	NVIDIA. Published by NVIDIA Developer blog.
2026; code	CompileIQ documentation	NVIDIA. Published by NVIDIA docs.
2025; company	Modular: What about the MLIR compiler infrastructure?	Modular. Published by Modular blog.
2026; company	Anthropic: Building a C compiler with a team of parallel Claudes	Anthropic. Published by Anthropic Engineering.
2026; company	Ars Technica: Sixteen Claude agents created a C compiler	Ars Technica (covering Anthropic). Published by Ars Technica.
2026; company	Modular/Lattner: The Claude C Compiler — Future of Software	Modular. Published by Modular blog.
2026; forum	HN: We tasked Opus 4.6 agent teams to build a C Compiler	Hacker News community. Published by Hacker News.
2026; company	Advancing GPU Programming with the CUDA Tile IR Backend for OpenAI Triton	NVIDIA. Published by NVIDIA Developer blog.

Forums & workshops

Date and type	Source	Organization and publisher
2025; forum	LLVM Discourse: LLVM ♥ ML Workshop 2025 agenda	LLVM community. Published by LLVM Discourse.
2026; forum	LLVM Discourse: LLVM ♥ ML Workshop 2026 CFP	LLVM community. Published by LLVM Discourse.
2023; forum	LLVM Discourse: ML-Guided Compiler Optimization Workshop 2023	LLVM community. Published by LLVM Discourse.
2026; forum	LLVM Discourse: Automated review with agents (~30 bugs / 207 PRs)	LLVM community. Published by LLVM Discourse.
2026; forum	comp.compilers: Magellan paper notice	comp.compilers community. Published by comp.compilers.
2025; company	IEEE Pulse: LLMs in Compiler Optimization — Challenges and Future Direction	IEEE EMBS Pulse. Published by IEEE Pulse.
2026; company	Moonlight literature review: Magellan	Moonlight. Published by Moonlight.
2025; forum	LLVM Discourse RFC: EmitC support for MLGO	Google / LLVM community. Published by LLVM Discourse.
2026; forum	LLVM MLGO sync minutes: EmitC path PoR (June 8, 2026)	Google / LLVM community. Published by LLVM Discourse.
2026; forum	LLVM Discourse RFC: mlir-opt-repl + MCP server	LLVM / MLIR community. Published by LLVM Discourse.

Correctness lineage

Date and type	Source	Organization and publisher
2010s+; code	Souper: A superoptimizer for LLVM IR	Google. Published by GitHub.

Source control & review agents

Date and type	Source	Organization and publisher
2026; paper	Archer: Towards Agentic Review for Compiler Optimizations	CUHK (cuhk-s3). Published by arXiv.
2026; paper	Agentic Harness for Real-World Compilers (llvm-harness)	SUSTech · ETH Zurich · CUHK. Published by arXiv.
2026; code	cuhk-s3/Archer (GitHub)	CUHK (cuhk-s3). Published by GitHub.
2024+; code	Gerrit plugin: ai-code-review (googlesource)	Google. Published by Gerrit googlesource.
2025+; code	amarula/reviewai-gerrit-plugin	Amarula Solutions. Published by GitHub.
2025+; code	GerritForge/ai-review-agent-provider	GerritForge. Published by GitHub.
2025; code	OpenEvolve (AlphaEvolve-style OSS)	Algorithmic SuperIntelligence. Published by GitHub.
2025; code	HeuriGym (cornell-zhang/heurigym)	Cornell University. Published by GitHub.

2025; code	meta-pytorch/KernelAgent	Meta (PyTorch). Published by GitHub.
2026; code	NVIDIA/CompileIQ (GitHub)	NVIDIA. Published by GitHub.
2026; code	anthropics/claudes-c-compiler	Anthropic. Published by GitHub.
2021+; code	google/ml-compiler-opt	Google. Published by GitHub.
2026; code	ZJU-PL/hintpilot	Zhejiang University (ZJU-PL). Published by GitHub.
ongoing; code	llvm/llvm-project (host repository)	LLVM Foundation / community. Published by GitHub.

Commercial products & proposals

Date and type	Source	Organization and publisher
2026; company	AMD ROCm: GEAK v3 kernel optimization agent	AMD. Published by AMD ROCm blog.
2026; code	GEAK v4: end-to-end GPU performance optimization system	AMD. Published by GitHub.
2026; company	GEAK Agent-Driven Optimization of the DeepSeekV4 MLA Kernel	AMD. Published by AMD ROCm blog.
2026; code	TensorRT-LLM PR: Claude agents/skills for kernels and compile	NVIDIA. Published by GitHub (TensorRT-LLM).
2026; company	CompileIQ docs: expected gains (2 to 3 percent highly optimized)	NVIDIA. Published by NVIDIA docs.
2026; company	CompileIQ agent-skills (AGENTS.md optimization campaigns)	NVIDIA. Published by NVIDIA CompileIQ docs.
2026; company	Hyperloom: A Multi-Agent Harness for Autonomous Inference Optimization on AMD GPUs	AMD. Published by AMD ROCm blog.

HW codesign & accelerator bring-up

Date and type	Source	Organization and publisher
2025/26; paper	Agentic Operator Generation for ML ASICs (TritorX)	Meta. Published by arXiv.
2025/26; paper	KernelEvolve: Scaling Agentic Kernel Coding for Heterogeneous AI Accelerators at Meta	Meta. Published by ISCA 2026 • arXiv.
2026; company	Meta Engineering blog: KernelEvolve / Ranking Engineer Agent	Meta. Published by Engineering at Meta.
2026; paper	Compiler-Grounded Hierarchical Diagnosis for Triton on NPUs	Huawei Technologies. Published by arXiv.
2026; paper	Rethinking Agentic Kernel Generation for Emerging Accelerators (Zomboss)	University of Michigan • The University of Texas at Austin. Published by arXiv.
2026; paper	KForge: Cross-Platform Kernel Generation for AI Accelerators	Gimlet Labs. Published by MLArchSys @ ISCA 2026 • arXiv.

Agent control-plane substrate

Date and type	Source	Organization and publisher
2026; paper	Auto: The AGI Compiler	RightNow AI. Published by arXiv.
2026; paper	FlowCompile: An Optimizing Compiler for Structured LLM Workflows	UMass Amherst • MIT • MIT-IBM Watson AI Lab. Published by arXiv.
2026; paper	AgentFlow: Building Agent Dependency Graphs for Static Analysis of Agent Programs	Huazhong University of Science and Technology • Macquarie University. Published by arXiv.
2025; paper	Efficient and Scalable Agentic AI with Heterogeneous Systems	Stanford • Gimlet Labs • Intel. Published by arXiv.
2026; paper	VibeServe: Can AI Agents Build Bespoke LLM Serving Systems?	University of Washington (SyFI). Published by arXiv.
2026; code	DeepSeek Harness (`dsh`)	DeepSeek AI. Published by GitHub.
2026; paper	SKILL.state: Scalable Long-Horizon Agent Skills	Google LLC • Purdue University. Published by arXiv.
2025–26; company	Agent Skills (`SKILL.md`) open specification	Anthropic (origin) • agentskills.io ecosystem. Published by agentskills.io / GitHub • Anthropic Engineering.
2026; paper	SIGIL: Compiling Agent Skills into Typed Harnesses	University of Michigan. Published by arXiv.
2026; paper	SkCC: Portable and Secure Skill Compilation for Cross-Framework LLM Agents	Sun Yat-sen University. Published by arXiv • ACM CAIS 2026 (AgentSkills'26).
2026; paper	SkVM: Compiling Skills for Efficient Execution Everywhere	Shanghai Jiao Tong University. Published by arXiv.
2026; paper	SkillSmith: Compiling Agent Skills into Boundary-Guided Runtime Interfaces	AetherHeart Tech • Renmin University of China • UC San Diego. Published by arXiv.
2026; company	KCoral: Lightweight Benchmark Server for Agentic GPU Programming	MLC community (CMU • Apache TVM). Published by MLC blog • GitHub.

Structured and distributed compilation

Date and type	Source	Organization and publisher

2025; paper	Mirage: A Multi-Level Superoptimizer for Tensor Programs	Carnegie Mellon University • Peking University • Pennsylvania State University • Purdue University • Weizmann Institute of Science. Published by USENIX OSDI 2025.
2026; paper	Prism: Symbolic Superoptimization of Tensor Programs	Carnegie Mellon University • Tsinghua University • Weizmann Institute of Science. Published by arXiv.
2025; paper	Triton-distributed: Programming Overlapping Kernels on Distributed AI Systems with the Triton Compiler	ByteDance Seed • Tsinghua University • Peking University • Shanghai Jiao Tong University • Zhejiang University. Published by arXiv.
2026; code	Shardy: A Tensor Partitioning System	OpenXLA (GSPMD and PartIR teams). Published by OpenXLA documentation.
2026; paper	DITRON: Distributed Multi-level Tiling Compiler for Parallel Tensor Programs	ByteDance Seed • Peking University • Tsinghua University • Zhejiang University • Shanghai Jiao Tong University. Published by arXiv.

Validation and approximate hardware

Date and type	Source	Organization and publisher
2023; paper	Using the IBM Analog In-Memory Hardware Acceleration Kit for Neural Network Training and Inference	IBM Research. Published by APL Machine Learning • arXiv.
2026; paper	Measuring the Checker: Mutation Analysis for GPU-Kernel Benchmark Oracles	National University of Singapore • Nanyang Technological University • VinUniversity. Published by arXiv.

Controller improvement

Date and type	Source	Organization and publisher
2024/25; paper	Automated Design of Agentic Systems	University of British Columbia • Vector Institute. Published by ICLR 2025 • arXiv.
2024/25; paper	AFlow: Automating Agentic Workflow Generation	DeepWisdom • HKUST (Guangzhou) • Renmin University of China • Nanjing University • Fudan University • KAUST • Université de Montréal / Mila • HKUST. Published by ICLR 2025 • arXiv.
2025/26; paper	GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning	UC Berkeley • Stanford University • BespokeLabs.ai • University of Notre Dame • Databricks • MIT. Published by ICLR 2026 • arXiv.
2025; paper	A Self-Improving Coding Agent	University of Bristol • iGent AI. Published by arXiv.
2025/26; paper	Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents	University of British Columbia • Vector Institute • Sakana AI. Published by ICLR 2026 • arXiv.
2026; paper	Hyperagents	University of British Columbia • Meta • Vector Institute • University of Edinburgh (among listed affiliations). Published by arXiv.
2026; paper	Meta-Harness: End-to-End Optimization of Model Harnesses	Stanford University • KRAFTON • MIT. Published by arXiv.
2026; paper	Beyond Scaling: Self-Evolving LLM Agents for Hardware Kernel Optimization via an Experience-Driven Workflow and Experience Graph Memory	Affiliations not verified in the accessible primary abstract. Published by arXiv.
2026; paper	MetaKernelBench: Measuring GPU Kernel Knowledge Transfer Beyond Code	Westlake University • Shanghai Jiao Tong University • Zhejiang University. Published by arXiv.

Total: 182 source digests.

Export notes

Source links open primary references; full digests remain in the repository. Related records are not independent replications. The presentation is an earlier snapshot. Rebuild with `python3 publish/build_pdf.py`.